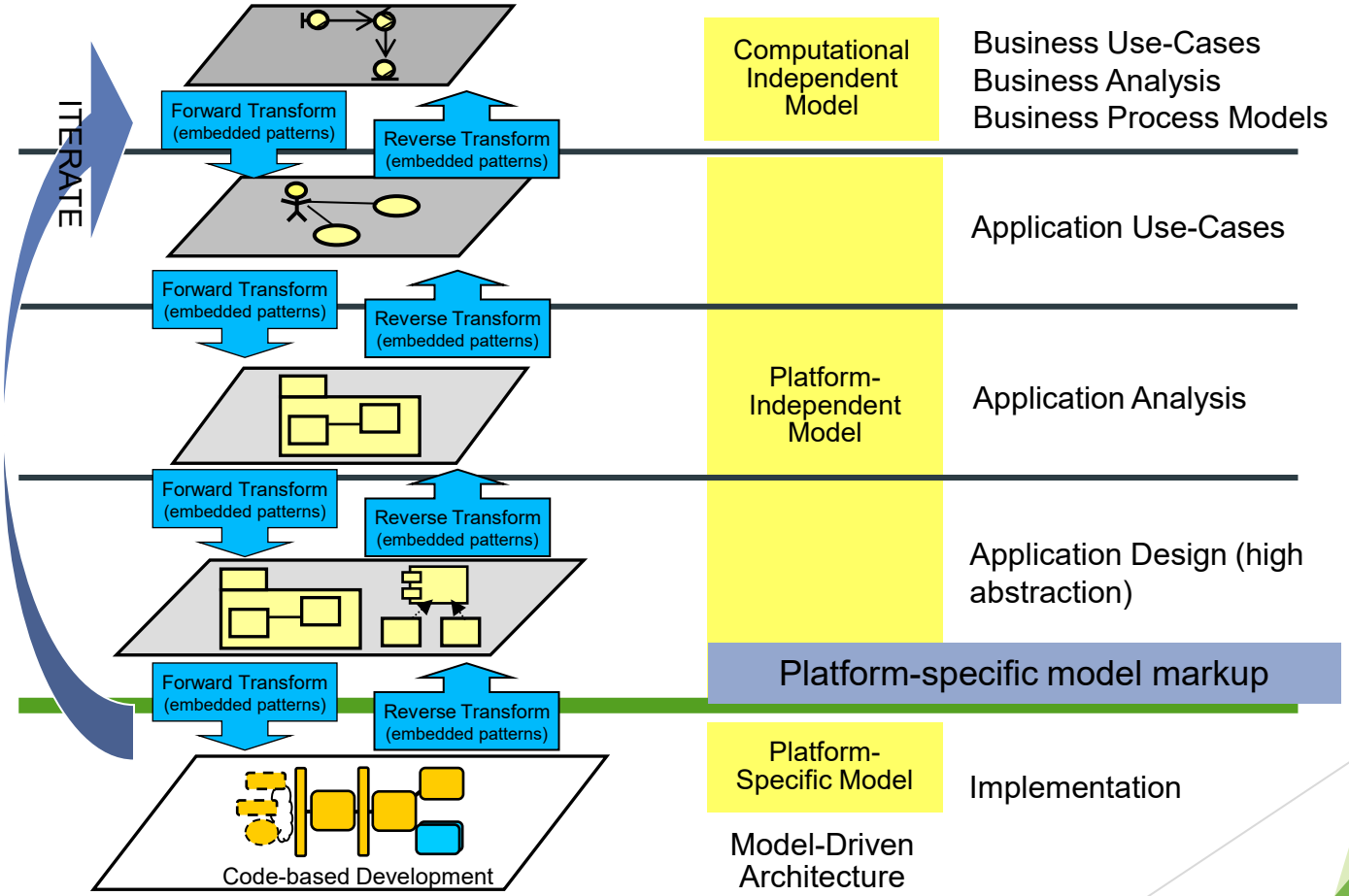
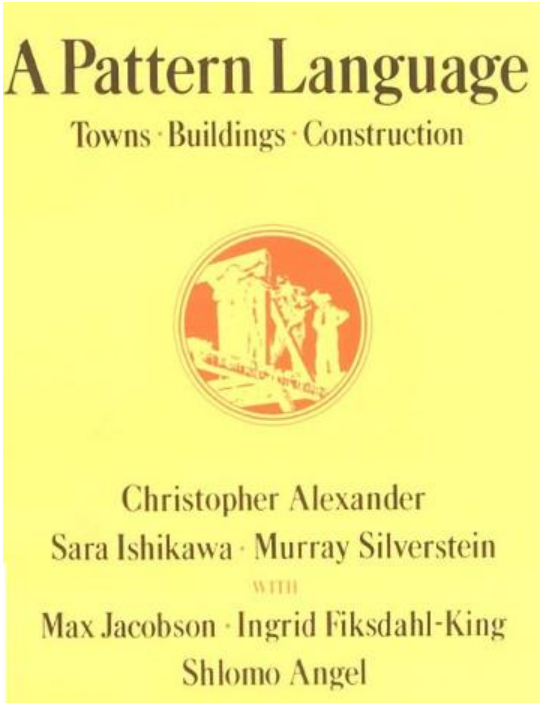
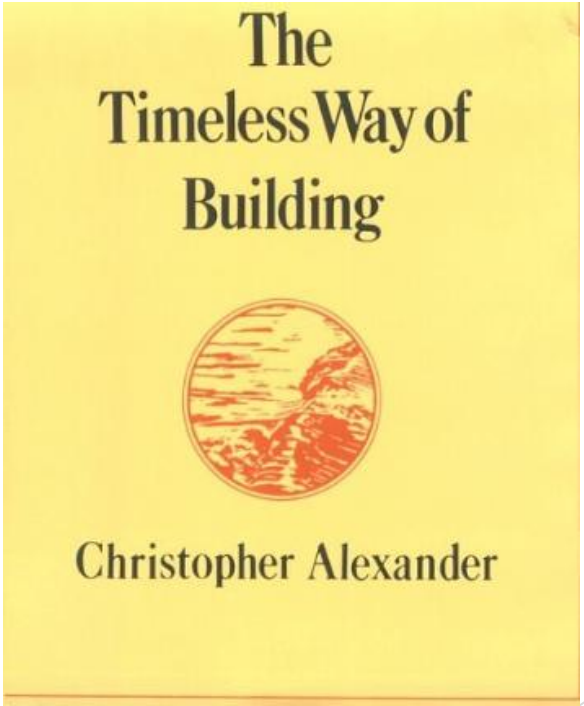

ITERATE
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Code-based Development
Computational Independent Model
Business Use-Cases
Business Analysis
Business Process Models
Application Use-Cases
Platform-Independent Model
Application Analysis
Application Design (high abstraction)
Platform-specific model markup
Platform-Specific Model
Implementation
Model-Driven Architecture

Los patrones. Origen

- Tienen su origen en un campo distinto al de la informática: La Arquitectura.
- Christopher Alexander recopila 253 patrones
- El objetivo era el crear entornos que redundaran en una ***quality without name*** *(*esencia de todas las cosas vivas y útiles que nos hacen sentir vivos, nos da satisfacción y mejora la condición humana*)*, que al final se traduce en calidad de vida de los residentes en un país, ciudad o edificio
- Para alcanzar la calidad, hay que atravesar **la puerta** (lenguaje común de patrones)
- A la puerta se llega a través de **un camino**

Christopher Alexander





Los patrones. Evolución

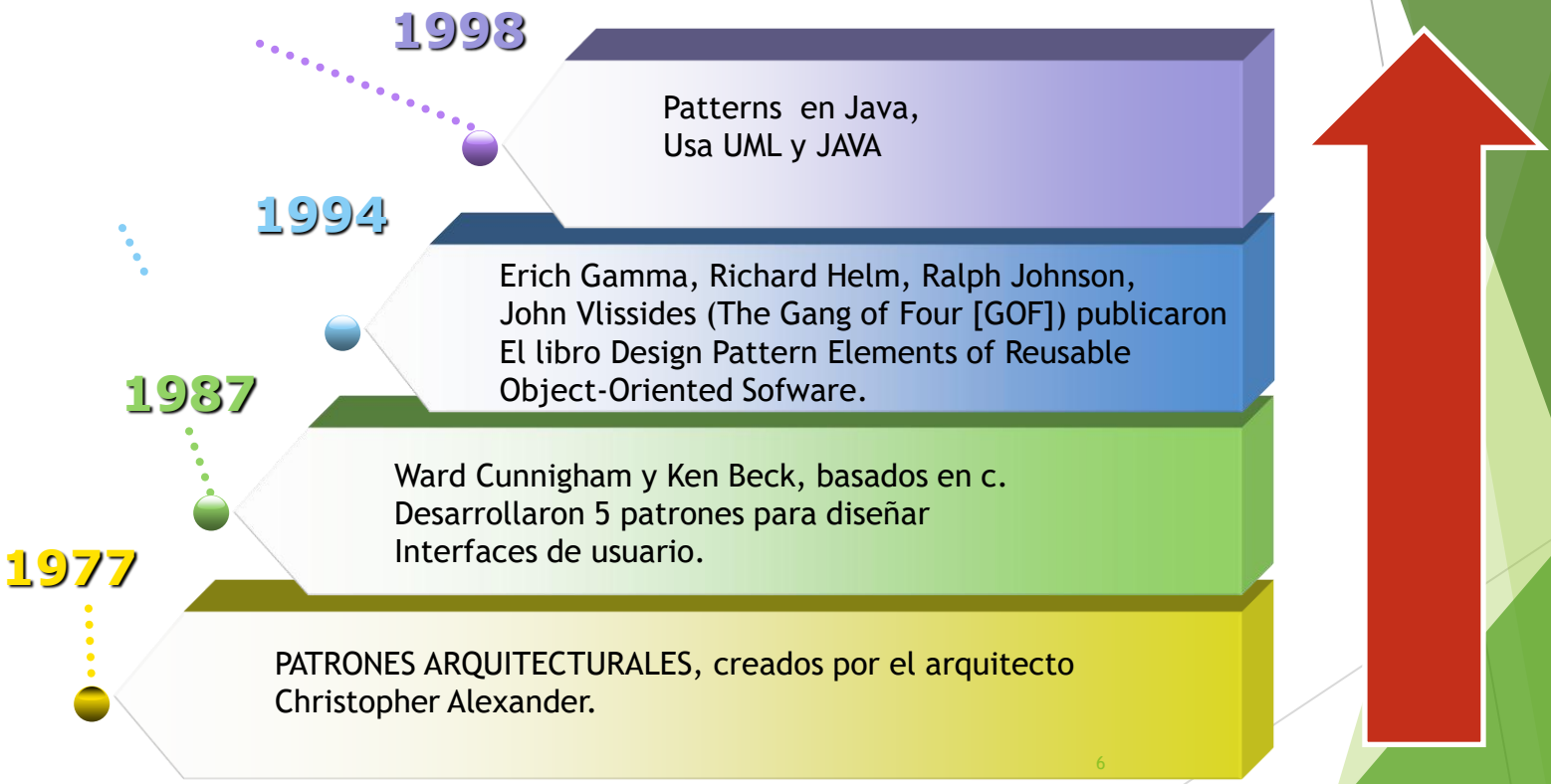
- Se utiliza inicialmente en el campo de la **Arquitectura** por Christopher Alexander. Mitad de los 70.
  - 1977, “A pattern language: Towns/Builder/Construction”
  - 1979, “The Timeless Way of Building”

- La idea se extrapola al terreno de la **Informática**
  - 1989, Cunningham y Beck -> Smalltalk
  - 1991, Coplien -> idioms (patrones en C++)
  - 1994, *Gang of Four* (GoF) -> Design Patterns
  - 1997, Brad Appleton
  - ...

Libro “Design Patterns”

Principios de la década de 1990 , los patrones de diseño tuvieron éxito en el mundo de la informática fue a partir de la publicación del libro “Design Patterns” escrito por el grupo **Gang of Four (GoF)** compuesto por Erich Gamma, Richard Helm, Ralph Johnson y John Vlisides.

Historia


1998
Patterns en Java,
Usa UML y JAVA
1994
Erich Gamma, Richard Helm, Ralph Johnson,
John Vlissides (The Gang of Four [GOF]) publicaron
El libro Design Pattern Elements of Reusable
Object-Oriented Sofware.
1987
Ward Cunnigham y Ken Beck, basados en c.
Desarrollaron 5 patrones para diseñar
Interfaces de usuario.
1977
PATRONES ARQUITECTURALES, creados por el arquitecto
Christopher Alexander.

Definición de Patrón por Christopher Alexander

*"Cada patrón **describe un problema** que ocurre una y otra vez en nuestro entorno, para describir después el núcleo de la solución a ese problema, de tal manera que esa solución pueda ser usada más de un millón de veces sin hacerlo ni siquiera dos veces de la misma forma."*

"Each pattern describes a problem wich occurs over and over again in our environment, and describes the core of the solution to that problem, in such a way that you can use this solution a millon times over, without ever doing it the same way twice "

“Design Patterns”: Qué son?

Los patrones de diseño son soluciones reusables de problemas recurrentes.

Los patrones de diseño son la base para la búsqueda de soluciones a problemas comunes

Un patrón de diseño resulta ser una solución a un problema de diseño.

Se aplican en un contexto particular

Patrones. Características generales

- **Solucionar un problema**: los patrones capturan soluciones, no sólo principios o estrategias abstractas
- **Ser un concepto probado**: capturan soluciones demostradas, no teorías o especulaciones
- **La solución no es obvia**: los mejores patrones generan una solución a un problema de forma indirecta
- **Describe participantes y relaciones entre ellos**: describen módulos, estructuras del sistema y mecanismos complejos

Características específicas de un patrón

Para que una solución sea considerada un patrón debe poseer ciertas características:

1. Deben resolver problemas específicos de diseño
2. Debe ser reutilizable, lo que significa que es aplicable a diferentes problemas de diseño.
3. Se puede incluir en diferentes soluciones (utilizadas más de una vez en diferentes sistemas)
4. No apuntan a un dominio especifico de aplicación.

Objetivos - Design Pattern

- Proporcionar catálogos de elementos **reusables** en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Reutilización

- Con la reutilización se consigue:
  - Reducción de tiempos
  - Disminución del esfuerzo de mantenimiento
  - Eficiencia
  - Consistencia
  - Fiabilidad
  - Protección de la inversión en desarrollos
- ¿Cómo reutilizar? Usando mecanismos como: **Componentes**, **Objetos distribuidos,** **Patrones de diseño,** **Patrones arquitectonicos**, **Frameworks**,
  “Se debe ser consumidor de reutilización antes de ser productor de reutilización”

Ventajas/beneficios

- **Aumento de la productividad**
  - Simplifica y acelera la construcción y pruebas de Software
  - Reduce drásticamente los tiempos de desarrollo de Software mediante la eliminación de tareas repetitivas.
  - Ofrece facilidad de uso para los principiantes
- **Improved software governance (mejora la governaza del Sw)**
  - Permite cumplir en forma consistente un buen diseño arquitectónico, como también permite cumplir con estándares de codificación.
- **Aumento de la Calidad**
  - Mayor calidad del producto final debido a un mayor nivel de consistencia.
  - Diseño previamente probado
- Increased openness (Aumento de la transparencia)
  - Menos dependencia de una herramienta específica, proveedor o plataforma.

Patrones. Facilidades

- Facilitan la comunicación interna
- Ahorran tiempo y experimentos inútiles
- Mejoran la calidad del diseño y la implementación
- Son como “normas de productividad”
- Facilitan el aprendizaje de los paquetes de los lenguajes (por ej. Java)

Los patrones de diseño

- Los diseñadores expertos no resuelven cada problema desde el principio
- Se pueden encontrar patrones de clases y comunicaciones entre objetos en muchos sistemas orientados a objetos
- Los patrones de diseño ayudan a los diseñadores a reutilizar con éxito diseños para obtener nuevos diseños
- El objetivo de los patrones es guardar la **experiencia** en diseños de programas orientados a objetos
- Los **patrones de diseño** hacen más fácil reutilizar con éxito los diseños y arquitecturas, ayudan a elegir entre diseños alternativos, hacen a un sistema reutilizable y evitan alternativas que comprometen la **reutilización**
- **23 Patrones de diseño GOF**

Estructura de Patrones (descripción)

- Nombre
- Objetivo (Problema)
- Contexto
- Aplicabilidad (Fuerzas)
- Solución (Estructura)
- Consecuencias (pros y contras)
- Implementación
- Código del ejemplo (usos)
- Patrones relacionados, derivados o implicados

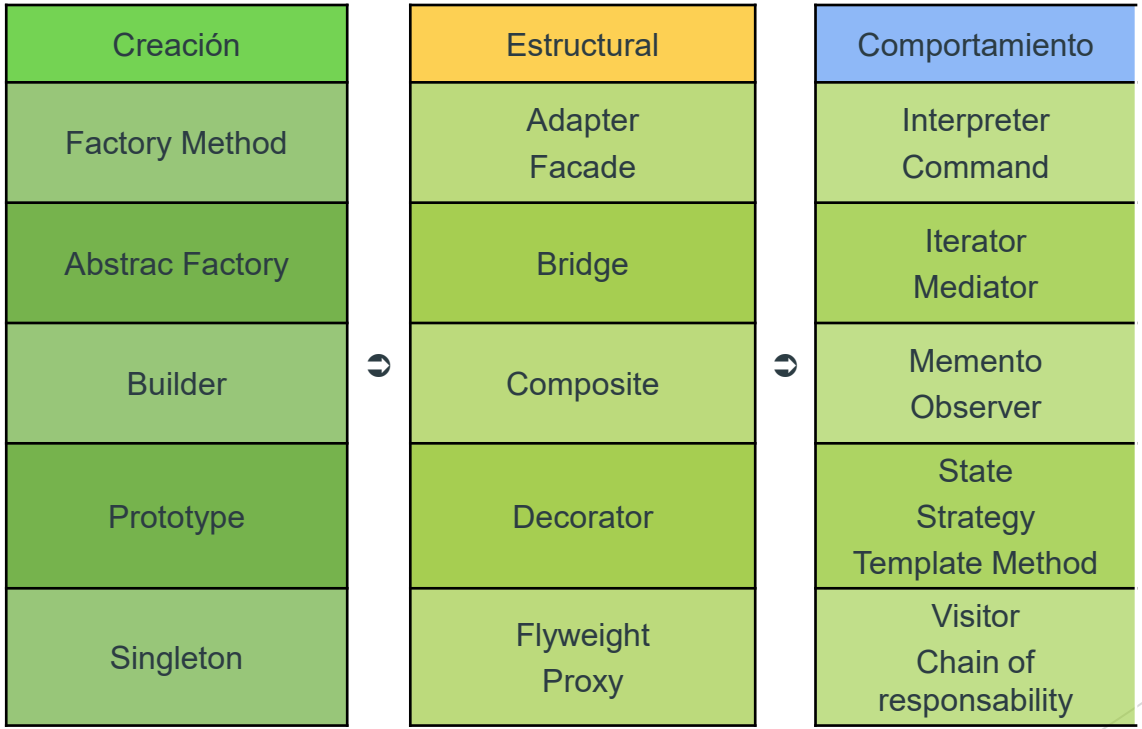
Tipos de patrones de diseño

- Patrones de **creación**. Procuran independizar al sistema de como sus objetos son creados y/o representados.
  Muestran la guía de cómo crear objetos cuando sus creaciones requieren tomar decisiones. Estas decisiones normalmente serán resueltas dinámicamente decidiendo que clases instanciar o sobre que objetos un objeto delegará responsabilidades.

- Patrones **estructurales**. Describen la forma en que diferentes clases y tipos de objetos pueden ser organizados para trabajar unos con otros, generalmente en formas mas complejas.

- Patrones de **comportamiento**. Se utilizan para organizar, manejar y combinar comportamientos.
  Se centran en los algoritmos y en la asignación de responsabilidades entre los objetos.

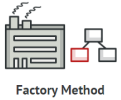
Clasificación de los 23 patrones (GOF)

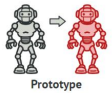


Patrones creacionales





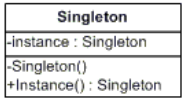






- Abstract Factory (fábrica abstracta): permite trabajar con objetos de distintas familias de manera que las familias no se mezclen entre sí y haciendo transparente el tipo de familia concreta que se esté usando.
- Builder (constructor virtual): abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto.
- Factory Method (método de fabricación): centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística para elegir el subtipo que crear.
- Prototype (prototipo): crea nuevos objetos clonándolos de una instancia ya existente.
- Singleton (instancia única): garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

SINGLETON



Singleton: Define un método Instance() que permite al cliente el acceso a su única instancia. El método Instance() es estático.

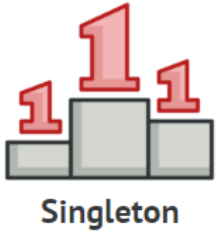
Esta misma clase es la responsable de la creación y mantenimiento de su propia instancia

Asegurar que solo existe una única instancia de una clase, y que hay un punto global de acceso a la misma.

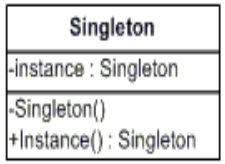
Aplicabilidad:

- Es necesario cuando hay clases que tienen que gestionar de manera centralizada un recurso.
- Asegurarse de que una clase tiene una sola instancia y proporcionar un punto de acceso global a ella. OJO con la **SINGLETONITIS**

SINGLETON








```
public class Singleton
{
    private static Singleton INSTANCE = new Singleton();

    // El constructor privado no permite que se genere un constructor por defecto
    // (con mismo modificador de acceso que la definición de la clase)

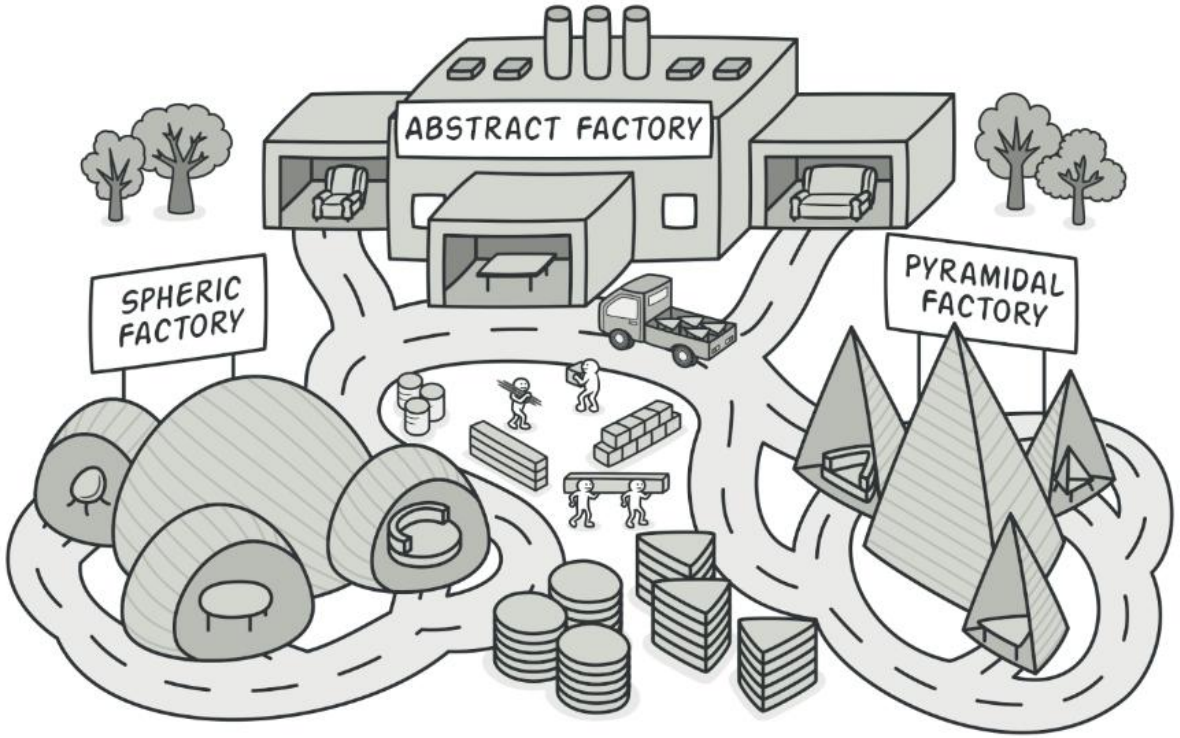
    private Singleton() { }

    public static Singleton getInstance() {       return INSTANCE;     }
}
```

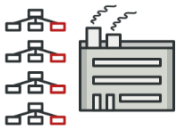
Abstract Factory

Abstract Factory

El patrón Abstract Factory nos permite crear, mediante una interfaz, conjuntos o familias de objetos (denominados productos) que dependen mutuamuente y todo esto sin especificar cual es el objeto concreto.

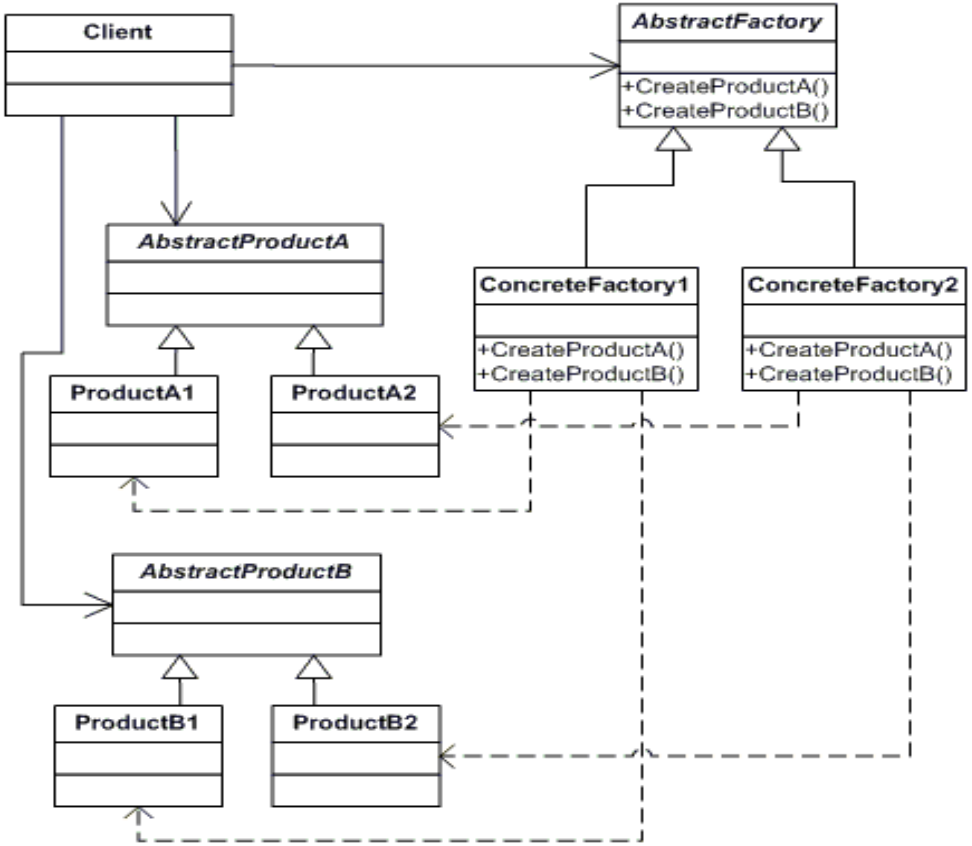


Abstract Factory



Se debe utilizar este patrón cuando:

- Un sistema se debe configurar con una de entre varias familias de productos.
- Una familia de productos relacionados están hechos para utilizarse juntos.



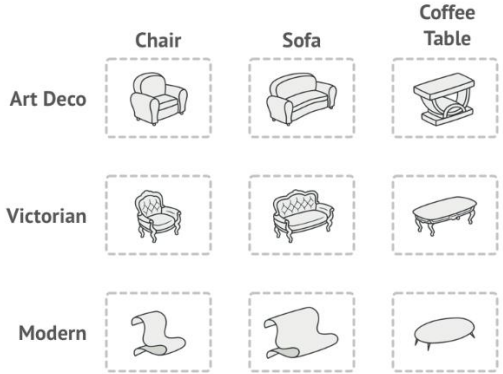
AbstractFactory: Declara una interfaz de operaciones que crean productos abstractos

ConcreteFactory: Implementa las operaciones que crean los objetos producto

AbstractProduct: Declara una interfaz para un tipo de objeto producto

Product: Define un objeto producto que será creado por el correspondiente ConcreteFactory

Client: Usa las interfaces



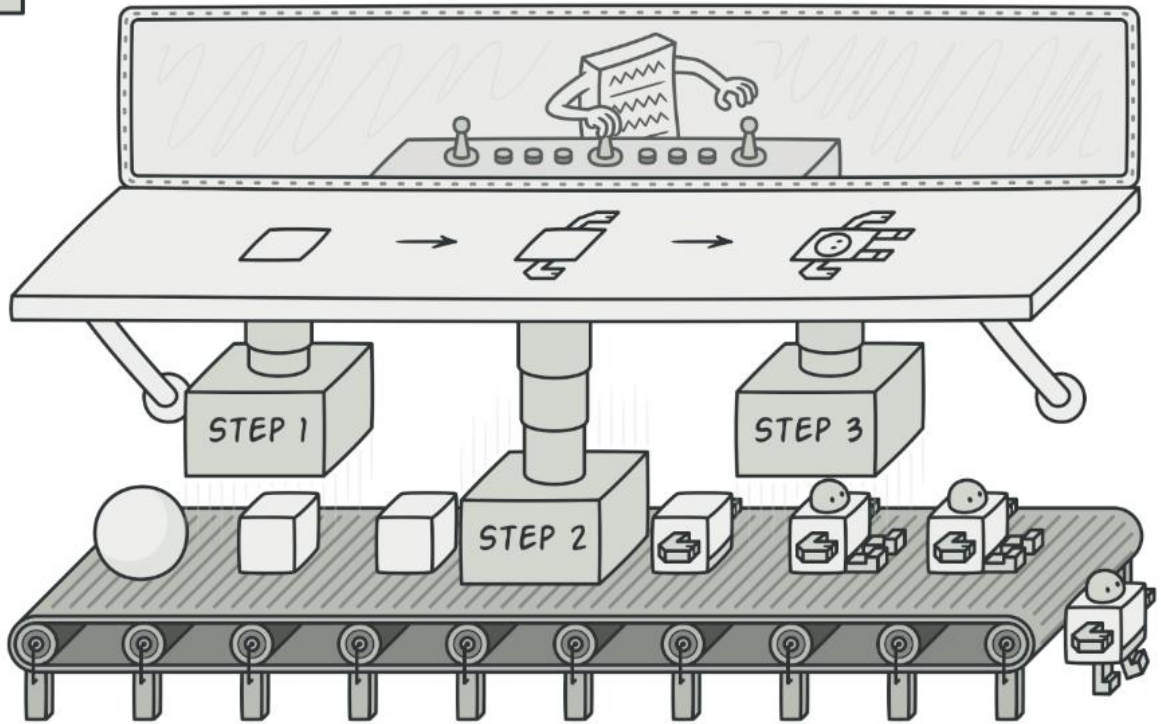
Product families and their variants.

Builder



Builder

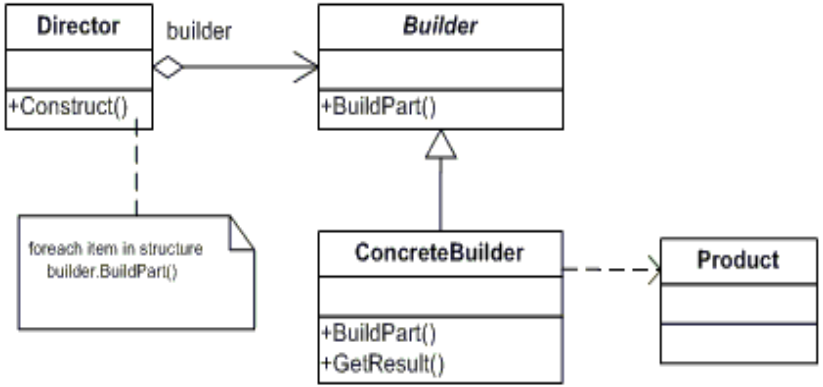
Como Patrón de diseño, el patrón builder (Constructor) es usado para permitir la creación paso a paso de una variedad de objetos complejos desde un objeto fuente (Producto).

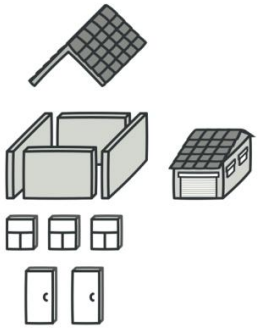


Builder

Se debe utilizar este patrón cuando sea necesario:

- Independizar el algoritmo de creación de un objeto complejo de las partes que constituyen el objeto y cómo se ensamblan entre ellas.
- Que el proceso de construcción permita distintas representaciones para el objeto construido, de manera dinámica.



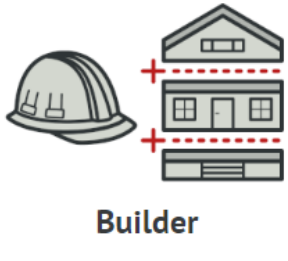


Builder:Define una interfaz para la creación de partes de un objeto complejo

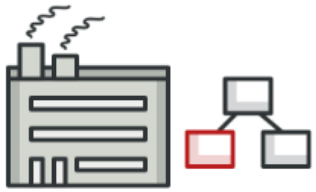
ConcreteBuilder: Implementa la interfaz de Builder, mantiene referencia del producto creado y proporciona una interfaz que retorna el producto. Ensambla las piezas constituyentes

Director:Construye un objeto usando la interfaz proporcionada por Builder

Product:Define las partes constituyentes del producto y representa un objeto complejo bajo construcción

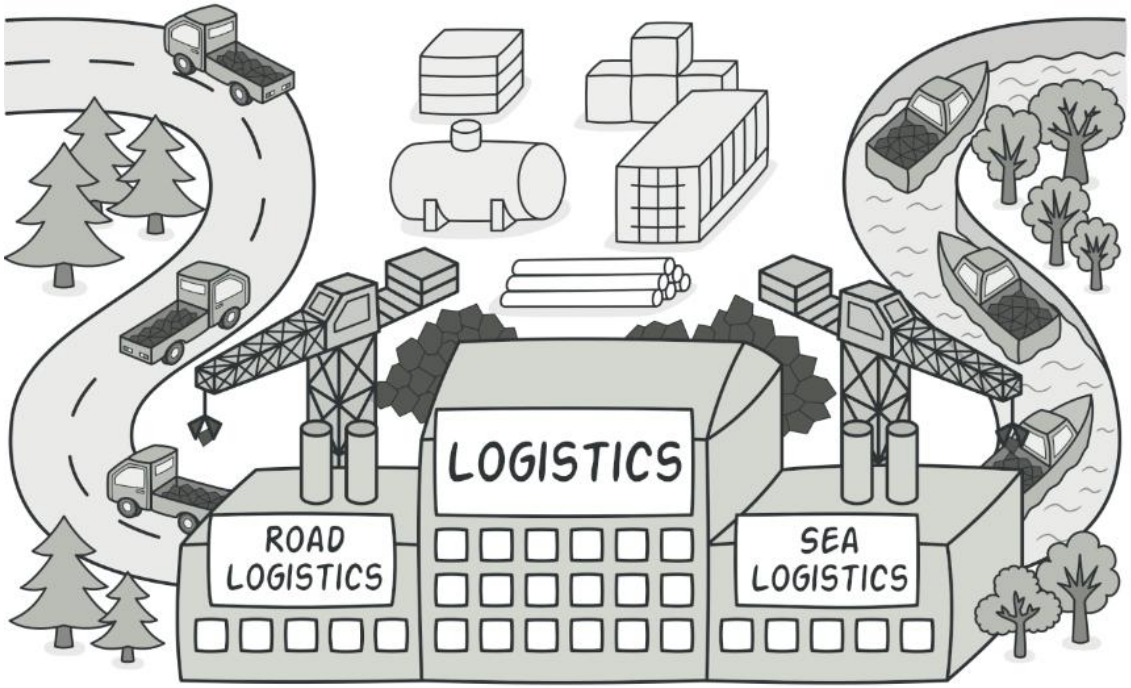


Factory Method



Factory Method

Consiste en utilizar una clase constructora abstracta con unos cuantos métodos definidos y otros abstractos.
Es una simplificación del Abstract Factory, en la que la clase abstracta tiene métodos concretos que usan algunos de los abstractos; según usemos una u otra hija de esta clase abstracta, tendremos uno u otro comportamiento.



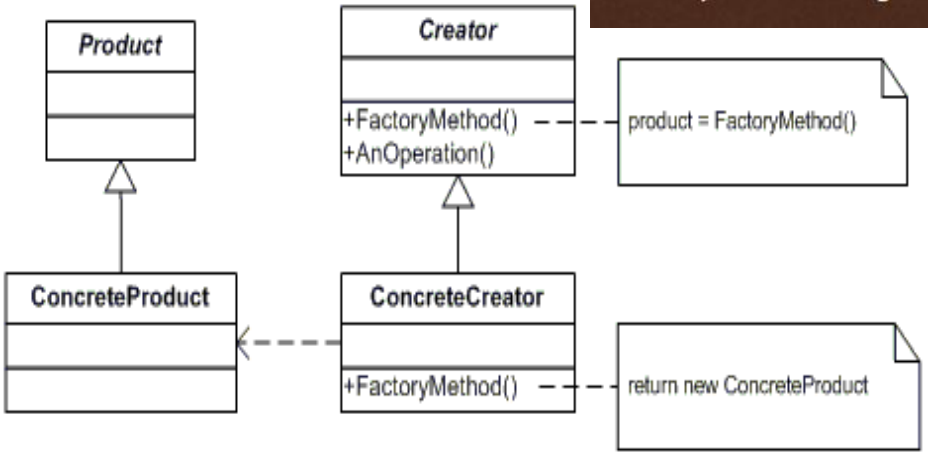
Factory Method



Factory Method

Este patrón debe ser utilizado cuando:

- Una clase no puede anticipar el tipo de objeto que debe crear y quiere que sus subclases especifiquen dichos objetos.
- Hay clases que delegan responsabilidades en una o varias subclases. Una aplicación es grande y compleja y posee muchos patrones creacionales.

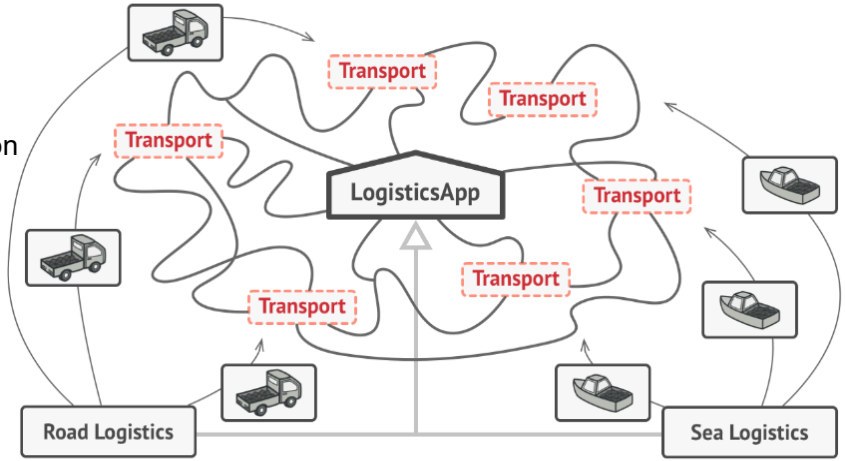


Creator:Declara el método FactoryMethod(). Se puede definir una implementación por defecto para él

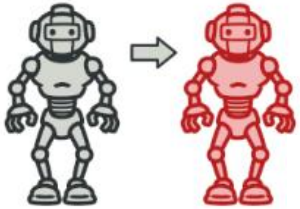
ConcreteCreator:Sobrescribe el método FactoryMethod(), devolviendo una instancia de ConcreteProduct

Product: Define la interfaz de objetos que crea el método FactoryMethod

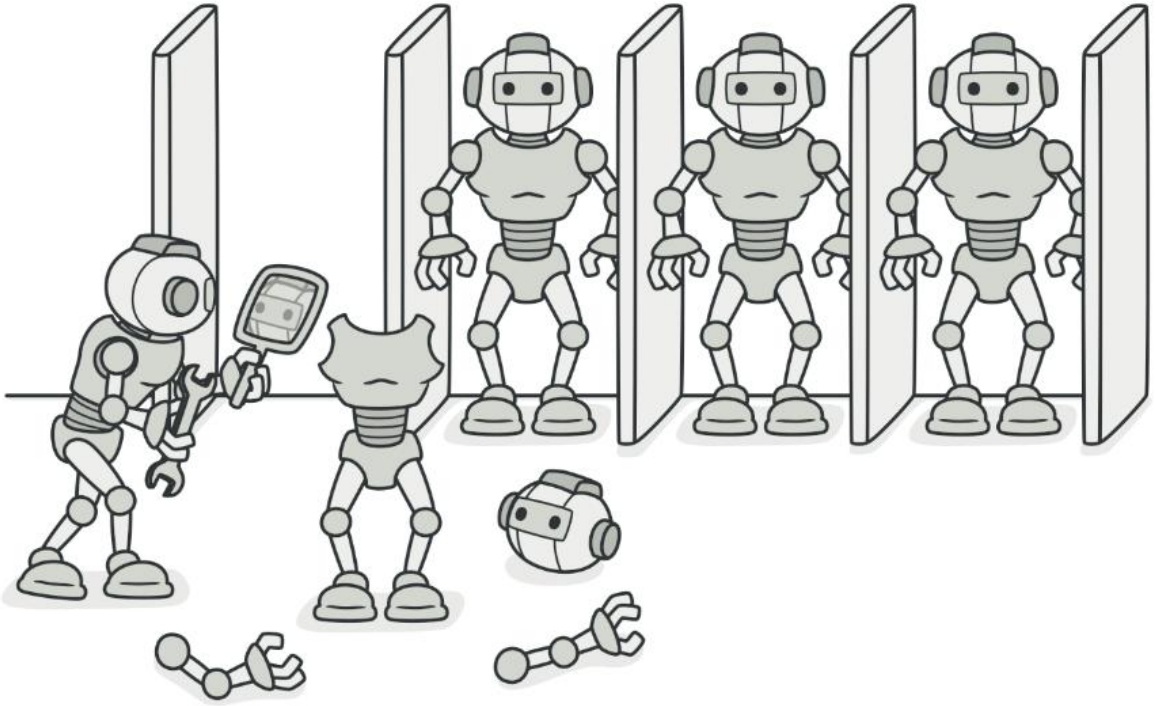
ConcreteProduct:Implementa la interfaz del producto



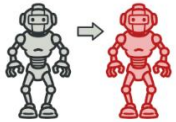
Prototype



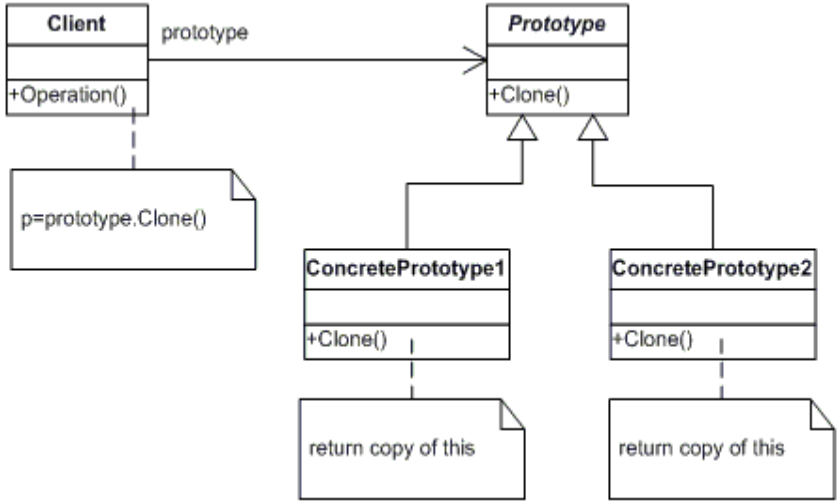
Tiene como finalidad crear nuevos objetos clonando una instancia creada previamente..



Prototype



Prototype



Cuando utilizar este patrón.

Aplica en un escenario donde sea necesario la creación de objetos parametrizados como "recién salidos de fábrica" ya listos para utilizarse, con la gran ventaja de la mejora de la performance: clonar objetos es más rápido que crearlos y luego setear cada valor en particular.

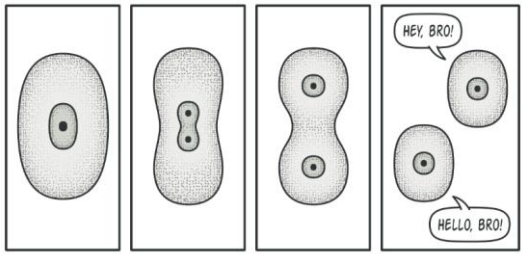
Este patrón debe ser utilizado cuando un sistema posea objetos con datos repetitivos, en cuanto a sus atributos: por ejemplo, si una biblioteca posee una gran cantidad de libros de una misma editorial, mismo idioma, etc. Hay que pensar en este patrón como si fuese un fábrica que tiene ciertas plantillas de ejemplos de sus prodcutos y, a partir de estos prototipos, puede crear una gran cantidad de productos con esas características.

Prototype: Declara una interfaz definiendo el método clone()

ConcretePrototype: implementa el método clone()

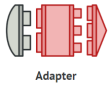
return (Prototype) this

Client:Crea un nuevo objeto pidiendo a un prototipo que se clone



The division of a cell.

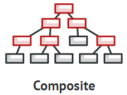
Patrones Estructurales



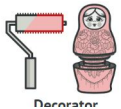
- Adapter (Adaptador): Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.



- Bridge (Puente): Desacopla una abstracción de su implementación.



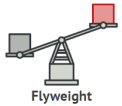
- Composite (Objeto compuesto): Permite tratar objetos compuestos como si se tratase de uno simple.



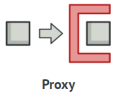
- Decorator (Envoltorio): Añade funcionalidad a una clase dinámicamente.



- Facade (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema.



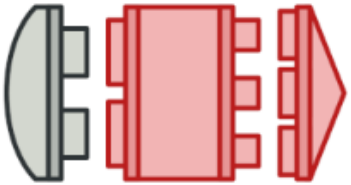
- Flyweight (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos que poseen idéntica información.



- Proxy: Mantiene un representante de un objeto.

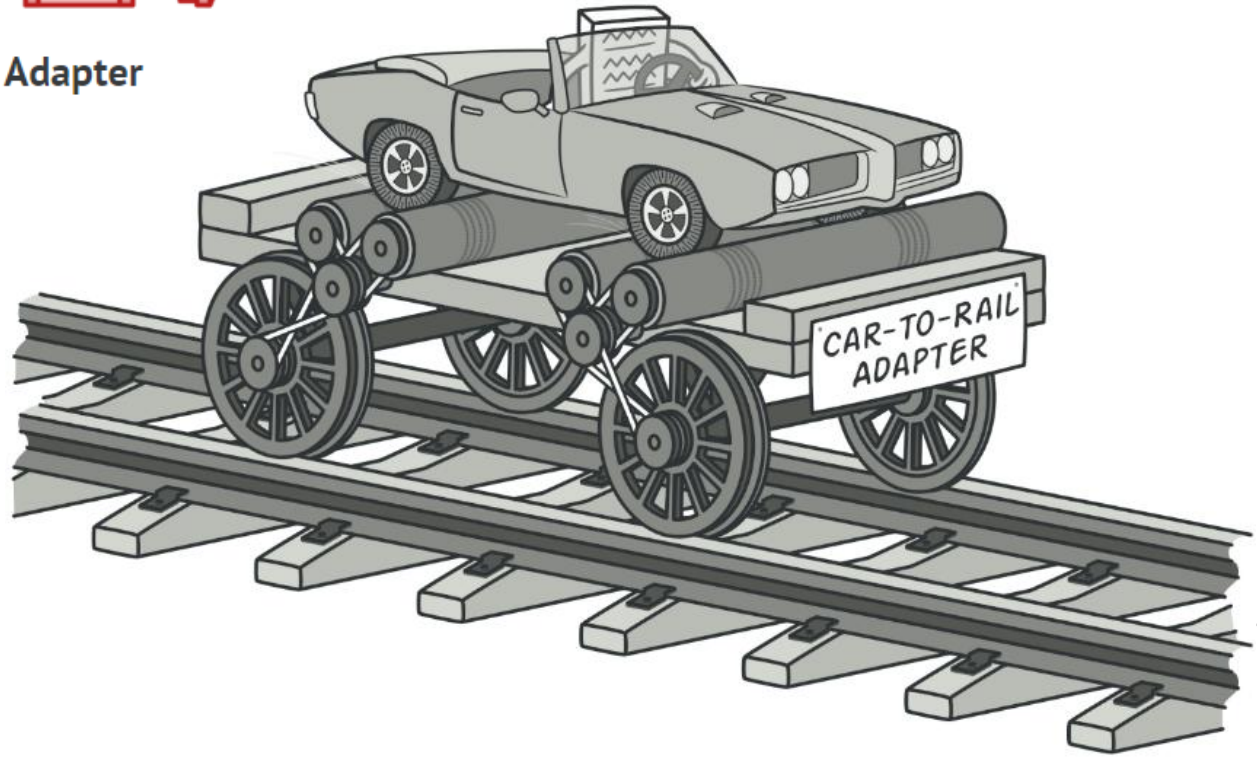
- Módulo (no es GOF): Agrupa varios elementos relacionados, como clases, singletons, y métodos, utilizados globalmente, en una entidad única.

Adapter

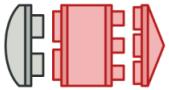


Adapter

Se utiliza para transformar una interfaz en otra, de tal modo que una clase que no pueda utilizar la primera haga uso de ella a través de la segunda. Actúa como un traductor



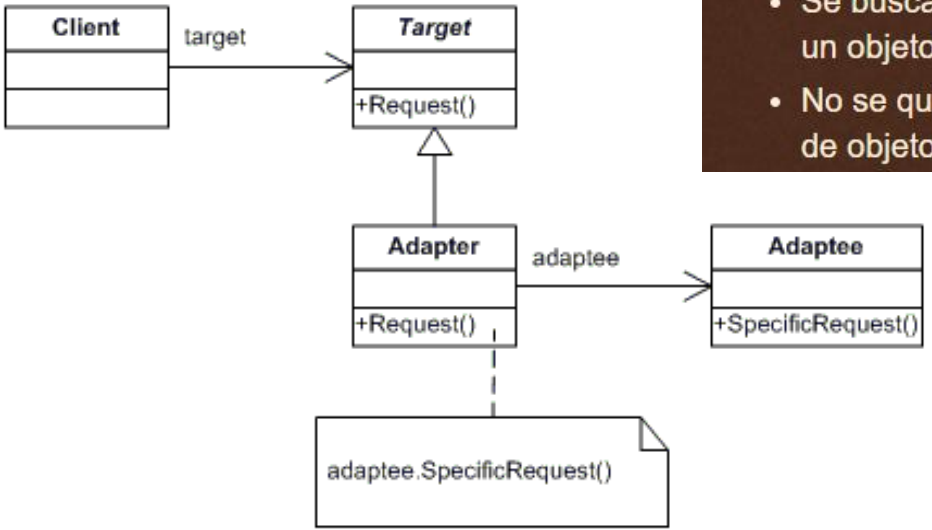
Adapter



Adapter

Este patrón se debe utilizar cuando:

- Se quiere utilizar una clase que llame a un método a través de una interface, pero se busca utilizarlo con una clase que no implementa ese interface.
- Se busca determinar dinámicamente que métodos de otros objetos llama un objeto.
- No se quiere que el objeto llamado tenga conocimientos de la otra clase de objetos.

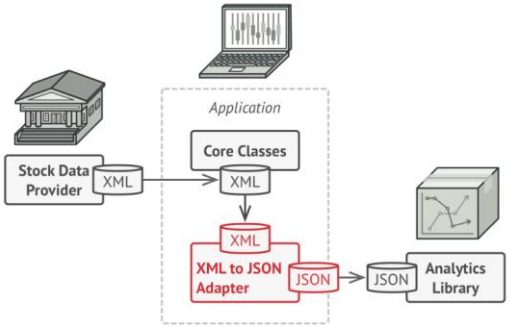


Target: Define una interfaz de dominio específico que el cliente utiliza

Adapter: Adapta interfaces de Adaptee y Target

Adaptee: Define una interfaz que existiendo necesita adaptación

Client: utiliza objetos ajustándose a la interfaz ofrecida por Target

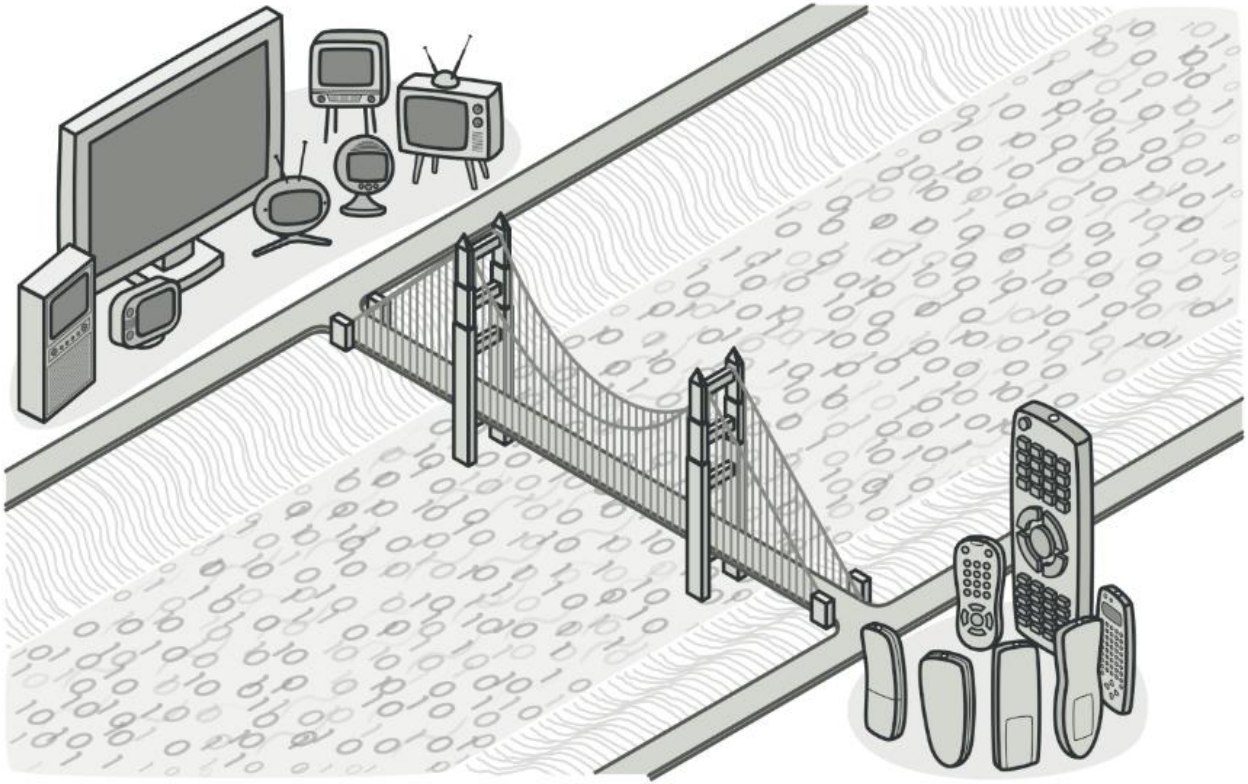


Bridge



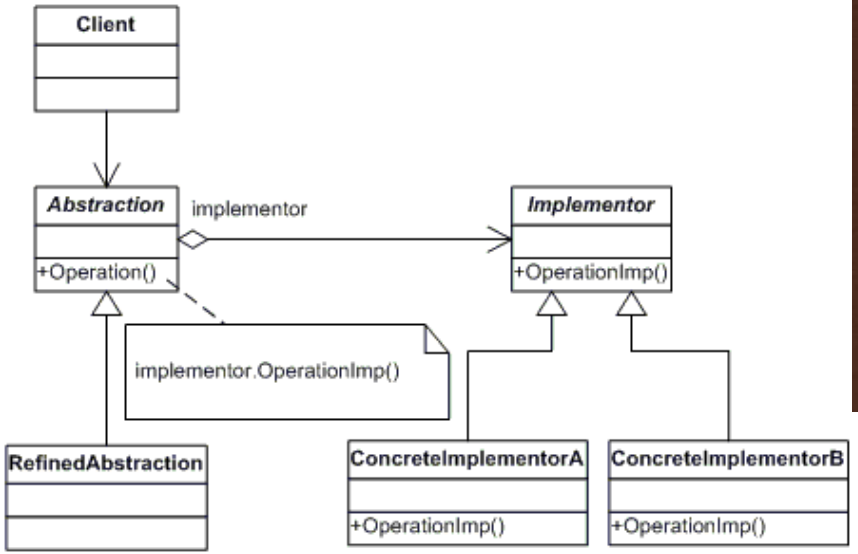
Bridge

Es una técnica usada en programación para desacoplar una abstracción de su implementación, de manera que ambas puedan ser modificadas independientemente sin necesidad de alterar por ello la otra.



Bridge





Este patrón debe ser utilizado cuando:

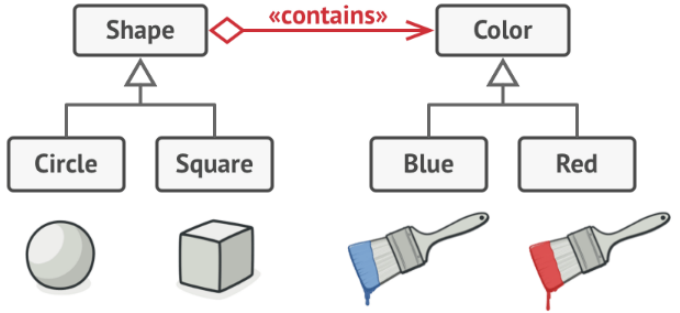
- Se desea evitar un enlace permanente entre la abstracción y su implementación. Esto puede ser debido a que la implementación debe ser seleccionada o cambiada en tiempo de ejecución.
- Tanto las abstracciones como sus implementaciones deben ser extensibles por medio de subclases. En este caso, el patrón Bridge permite combinar abstracciones e implementaciones diferentes y extenderlas independientemente.
- Cambios en la implementación de una abstracción no deben impactar en los clientes, es decir, su código no debe tener que ser recompilado.
- Se desea compartir una implementación entre múltiples y este hecho debe ser escondido a los clientes.
- Permite simplificar jerarquías demasiado pobladas.

Abstraction: define la interfaz de la abstracción

RefinedAbstraction: Extiende el interfaz definido por la abstracción

Implementor: define la interfaz de las clases de implementación, no tiene porque corresponderse con el interfaz de Abstraction. Aquí se suelen proporcionar primitivas, en Abstraction operaciones de más alto nivel

ConcreteImplementor: Implementa la interfaz de Implementor



COMPOSITE

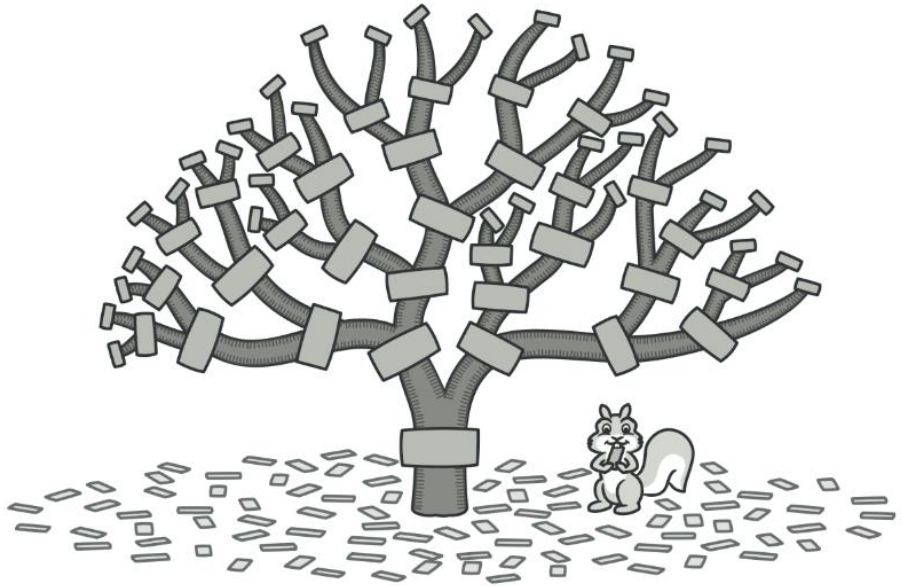


Composite

Sirve para construir objetos complejos a partir de otros más simples y similares entre sí, gracias a la composición recursiva y a una estructura en forma de árbol.

Aplicabilidad:

- Permite representar jerarquías de objetos similar a una estructura de árbol.
- Permite tratamiento uniforme de objetos simples y complejos, así como composiciones recursivas.
- Simplifica el código de los clientes, que sólo usan una interfaz.
- Facilita añadir nuevos componentes sin afectar a los clientes.
- Cuando se quiera que los clientes puedan ignorar la diferencia entre objetos simples y compuestos y tratarlos uniformemente

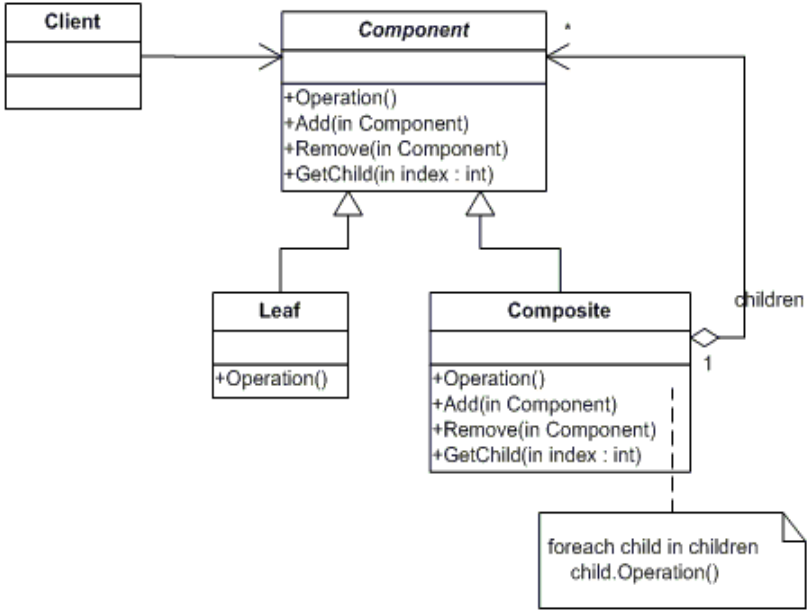


Composite



Se debe utilizar este patrón cuando:

- Se busca representar una jerarquía de objetos como “parte-todo”.
- Se busca que el cliente puede ignorar la diferencia entre objetos primitivos y compuestos (para que pueda tratarlos de la misma manera).

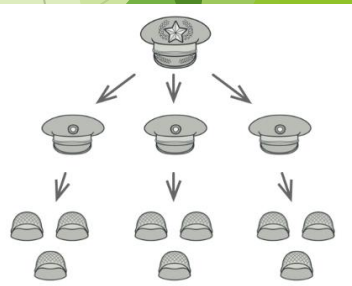


Component:declara la interfaz de los objetos involucrados en la composición, declara un interfaz para acceder y manejar sus hijos

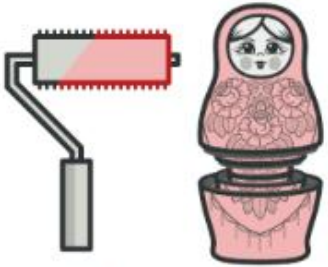
Leaf:representa objetos hoja en la composición.

Composite: define el comportamiento de los componentes que tienen hijos, almacena los componentes hijo, implementa operaciones relacionadas con los hijos

Client:manipula los objetos de la composición a través de Component


An example of a military structure.

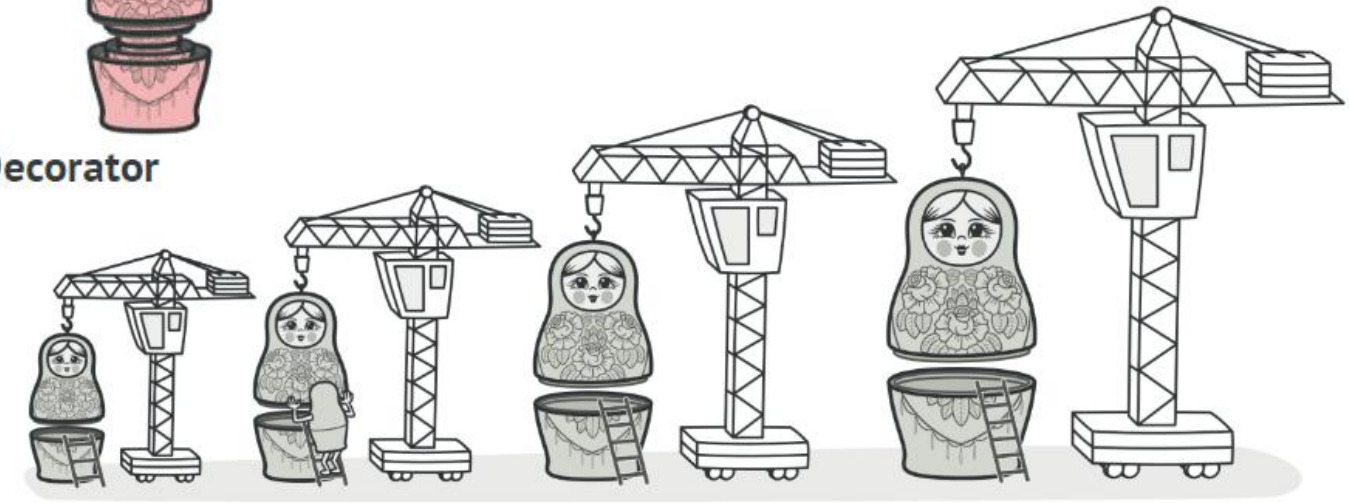
Decorator



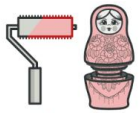
Decorator

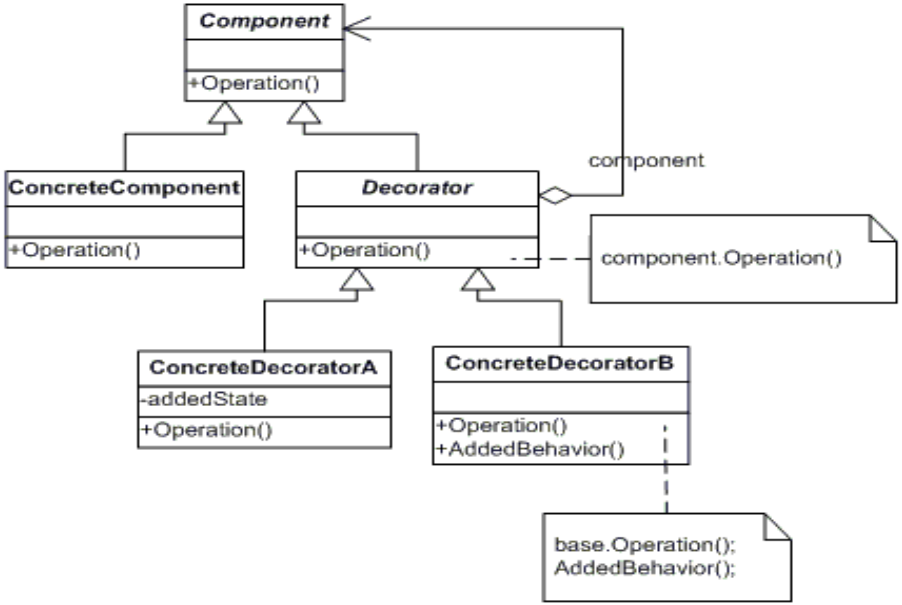
Este patrón se debe utilizar cuando:

- Hay una necesidad de extender la funcionalidad de una clase, pero no hay razones para extenderlo a través de la herencia.
- Se quiere agregar o quitar dinámicamente la funcionalidad de un objeto.



Decorator





Component:Define la interfaz que ofrecen los objetos que poseen responsabilidades añadidas dinámicamente

ConcreteComponent:Define un objeto al que responsabilidades adicionales pueden serle añadidas

Decorator: Mantiene referencia a un objeto Component y define una interfaz que conforma el interfaz del Component

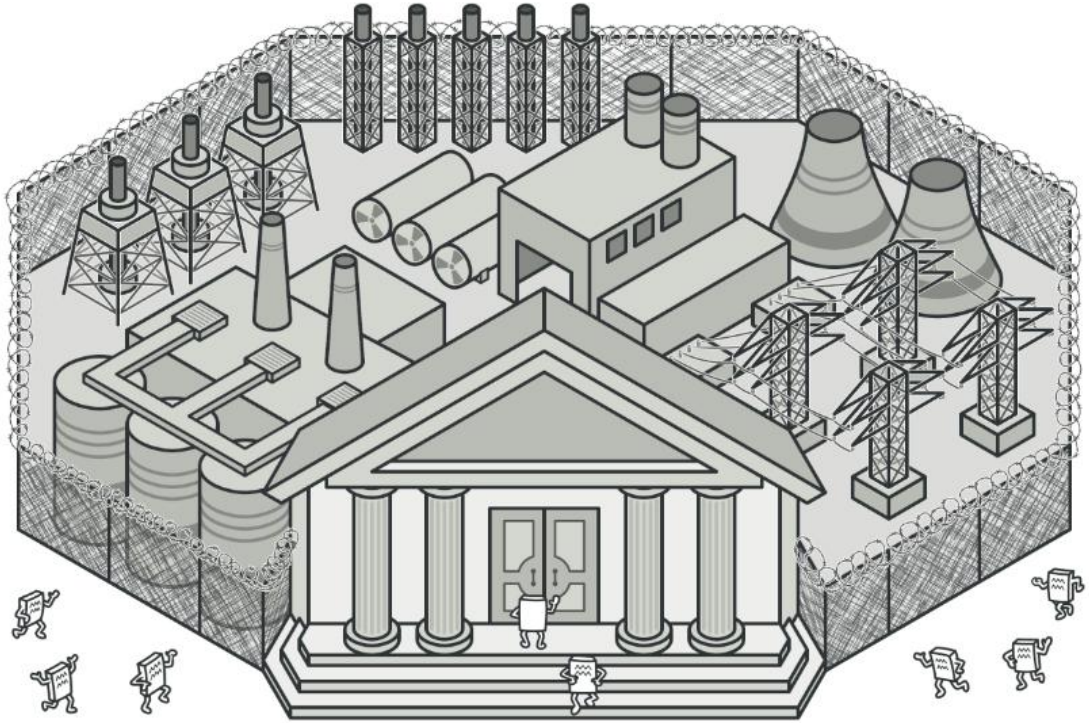
ConcreteDecorator:Añade las responsabilidades al Component

Facade



Facade

Viene motivado por la necesidad de estructurar un entorno de programación y reducir su complejidad con la división en subsistemas, minimizando las comunicaciones y dependencias entre estos.

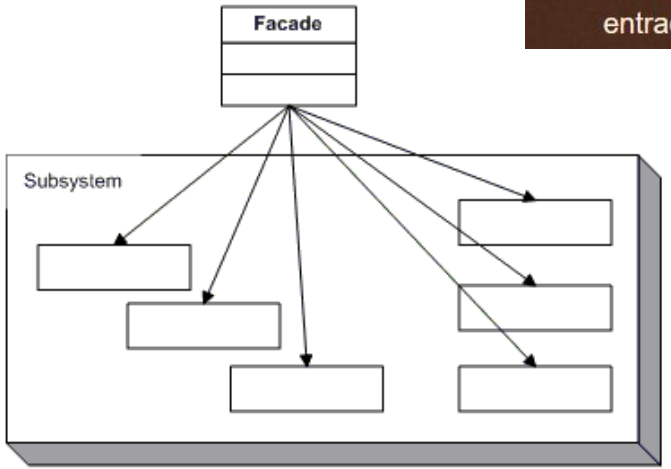


Facade



Se debe utilizar cuando:

- Se quiera proporcionar una interfaz sencilla para un subsistema complejo.
- Se quiera desacoplar un subsistema de sus clientes y de otros subsistemas, haciéndolo más independiente y portable.
- Se quiera dividir los sistemas en niveles: las fachadas serían el punto de entrada a cada nivel. Facade puede ser utilizado a nivel aplicación.

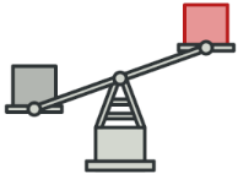


Facade::Conoce que clases subsistema son responsables de que peticiones y así, delega las peticiones a los objetos apropiados

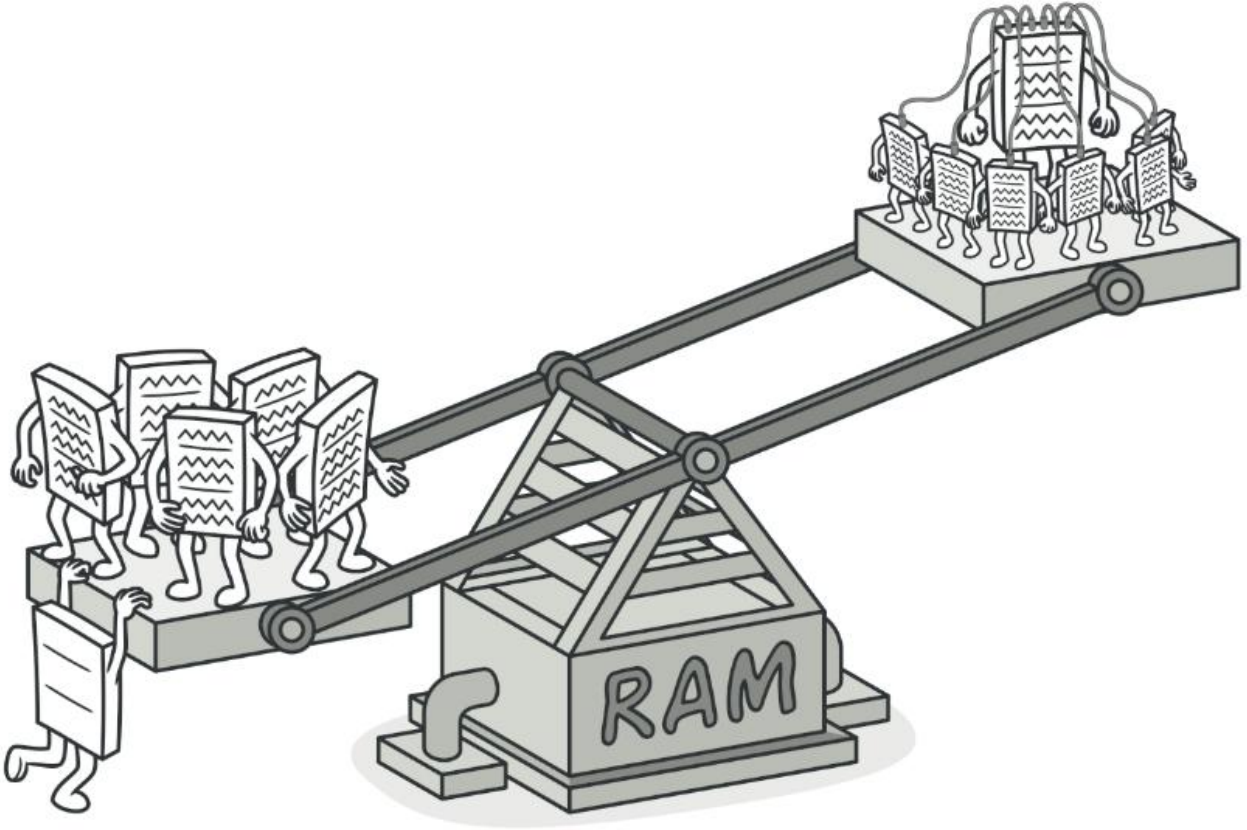
Subsystem:Implementa la funcionalidad del subsistema, realiza el trabajo solicitado por el objeto Facade y no conoce, ni mantiene referencia alguna del objeto Facade



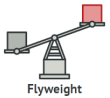
Flyweight

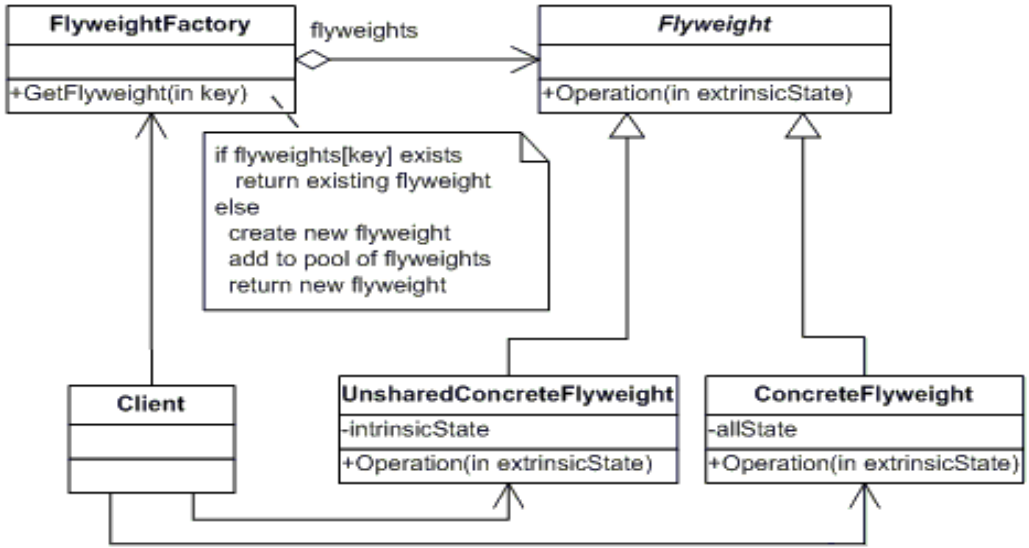


Sirve para eliminar o reducir la redundancia cuando tenemos gran cantidad de objetos que contienen información idéntica, además de lograr un equilibrio entre flexibilidad y rendimiento (uso de recursos).



Flyweight





Flyweight:declaran una interface a través de la que flyweights pueden recibir y actuar sobre estados externos

ConcreteFlyweight:

implementa la interfaz del flyweight y añade almacenamiento para el estado interno

UnsharedConcreteFlyweight: representa los objetos que no se repiten masivamente

FlyweightFactory:crea y gestiona los objetos Flyweight.

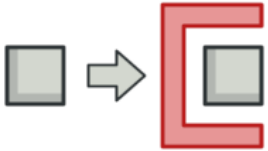
Client:mantiene una referencia a objetos flyweights

Este patrón se debe utilizar cuando:

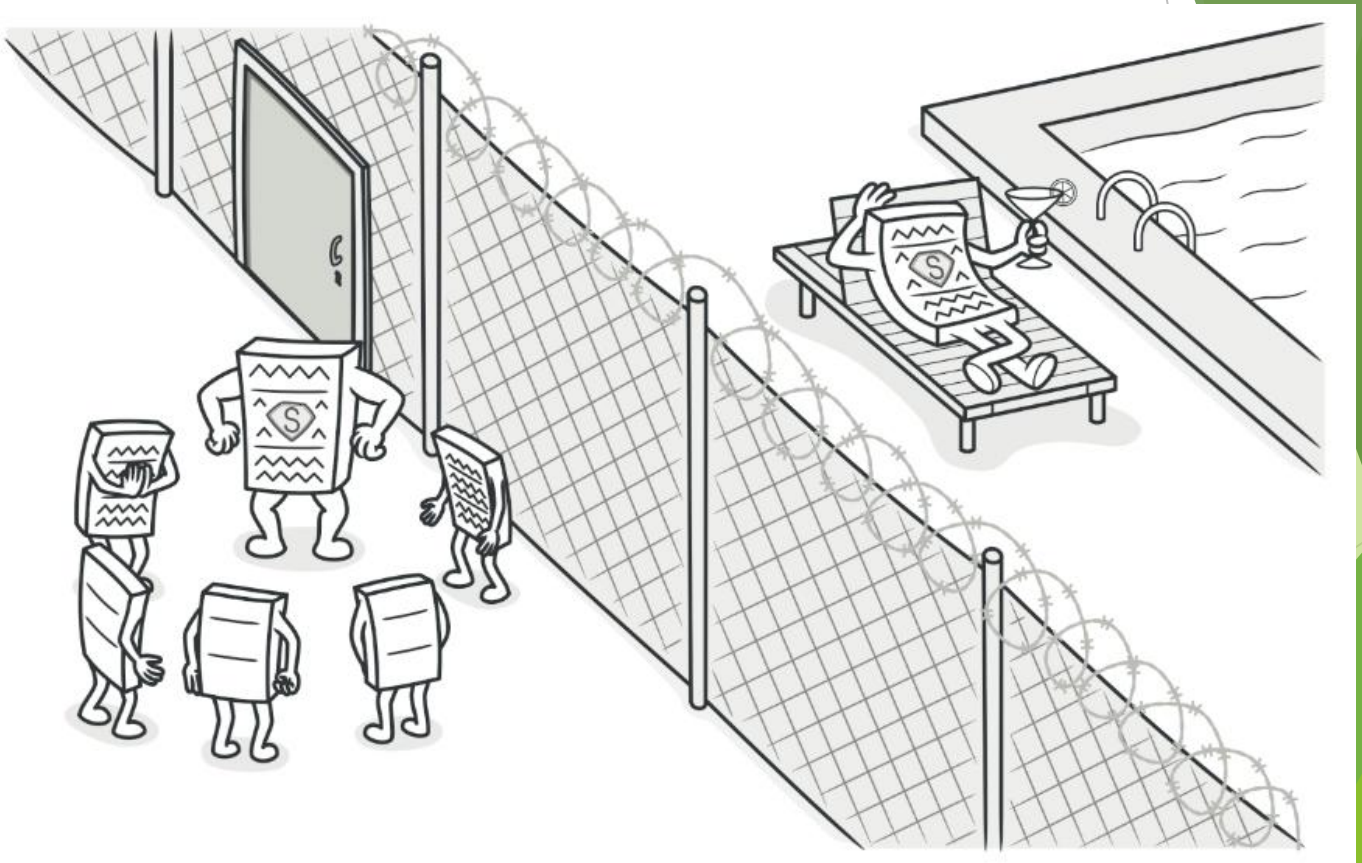
- Para eliminar o reducir la redundancia cuando se tiene gran cantidad de objetos que contienen la misma información.
- Cuando la memoria es crítica para el rendimiento de la aplicación.
- La aplicación no depende de la identidad de los objetos.

Proxy

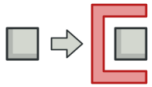
Tiene como propósito proporcionar un subrogado o intermediario de un objeto para controlar su acceso.

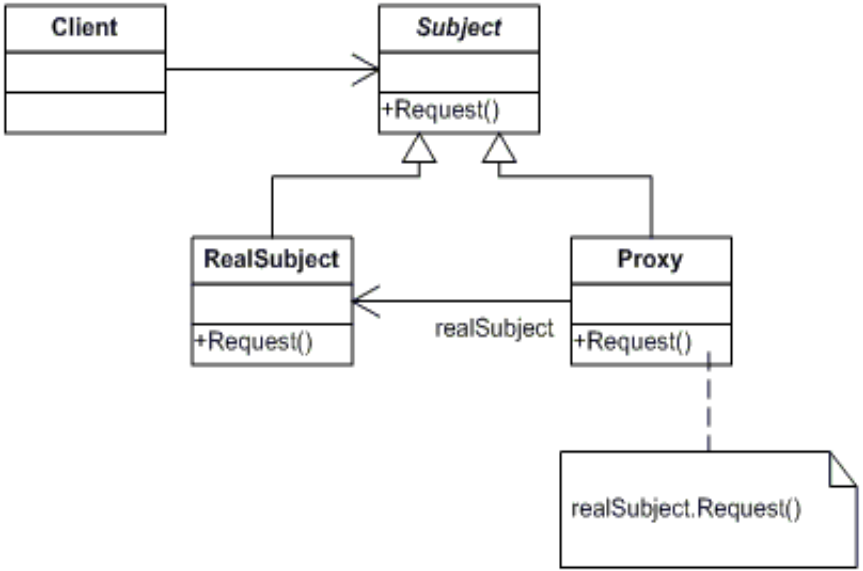


Proxy



Proxy





Cliente: interactua con el proxy , creyendo que es el objeto real

Subject:define una interfaz común para RealSubject y Proxy

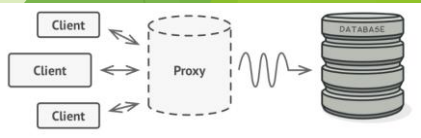
Proxy: intercepta la solicitud , ejecuta tareas (como seguridad) y luego decide si se lo pasa o no al objeto real.
Tiene referencia al objeto RealSubject, tiene una interfaz idéntica a la de Subject así un proxy puede sustituirse por un RealSubject, y controla el acceso al RealSubject y puede ser el responsable de su creación y borrado

RealSubject:define el objeto real que Proxy representa

Este patrón se debe utilizar cuando:

Se necesite retrasar el coste de crear e inicializar un objeto hasta que es realmente necesario.

- Se necesita una referencia a un objeto más flexible o sofisticada que un puntero.



Patrones Comportamiento



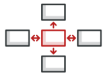
- Chain of Responsibility (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.



- Command (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- Interpreter (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.



- Iterator (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.



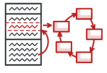
- Mediator (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.



- Memento (Recuerdo): Permite volver a estados anteriores del sistema.



- Observer (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.



- State (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.



- Strategy (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.



- Template Method (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.



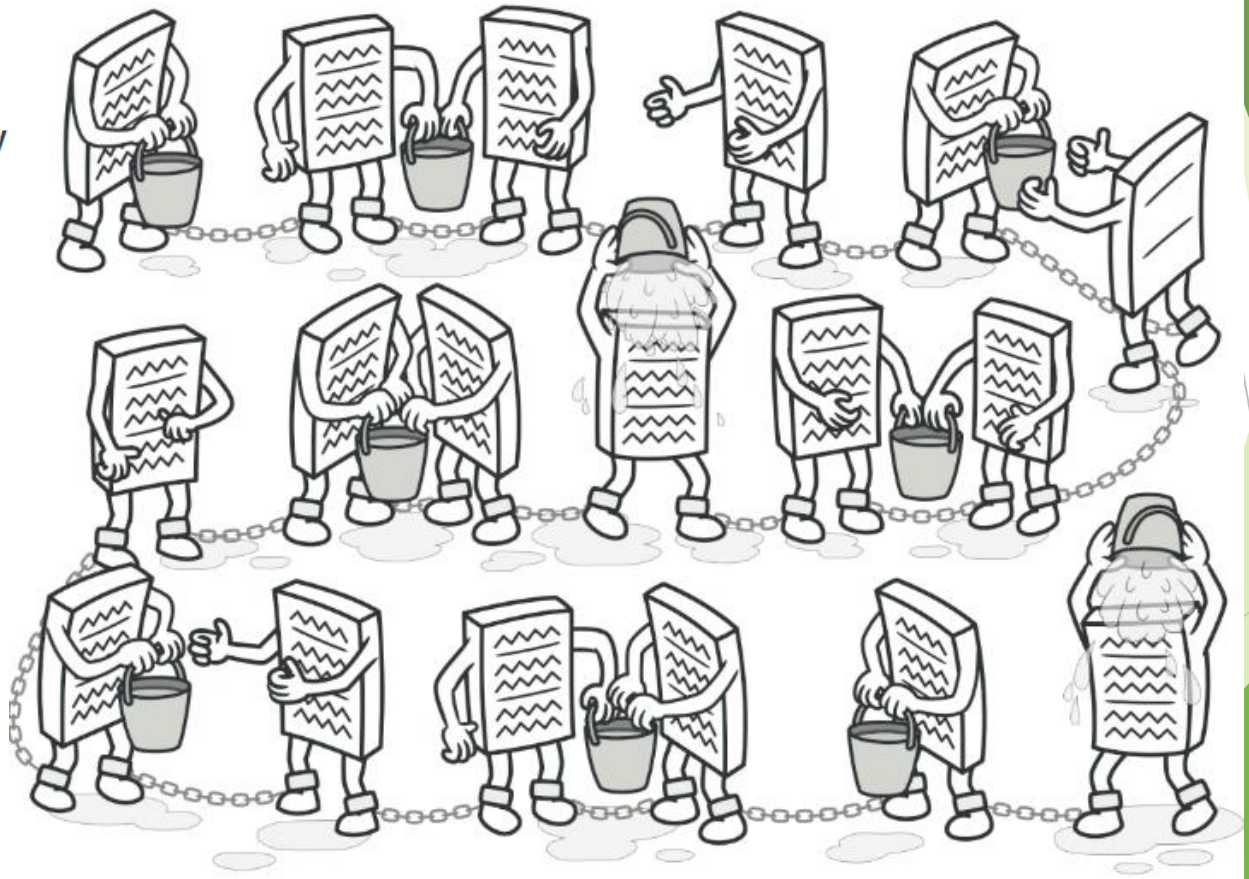
- Visitor (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

Chain of Responsability

Evita acoplar el emisor de una petición a su receptor dando a más de un objeto la posibilidad de responder a una petición.

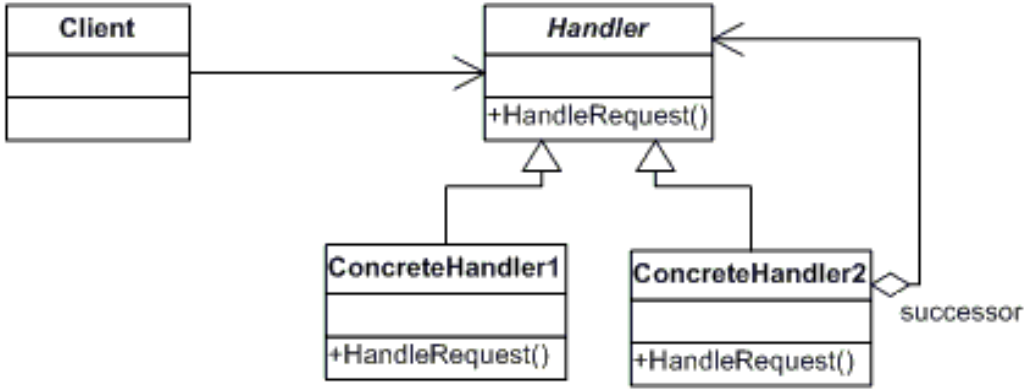


Chain of Responsibility



Chain of Responsability





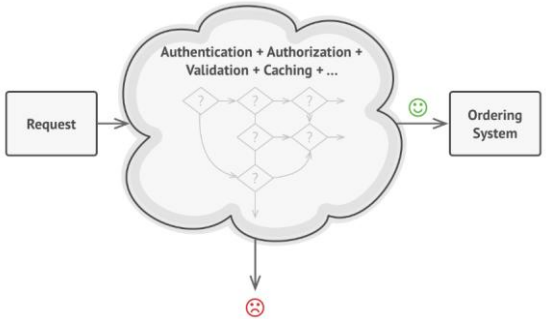
Client: hace las peticiones a los objetos ConcreteHandler pertenecientes a la cadena

Handler: define una interfaz para manejar las peticiones, opcionalmente implementa enlaces de sucesión de atención de tales peticiones

ConcreteHandler:recibe las peticiones y las atiende si es posible, de otra forma pasa la petición a su sucesor

Se utiliza cuando:

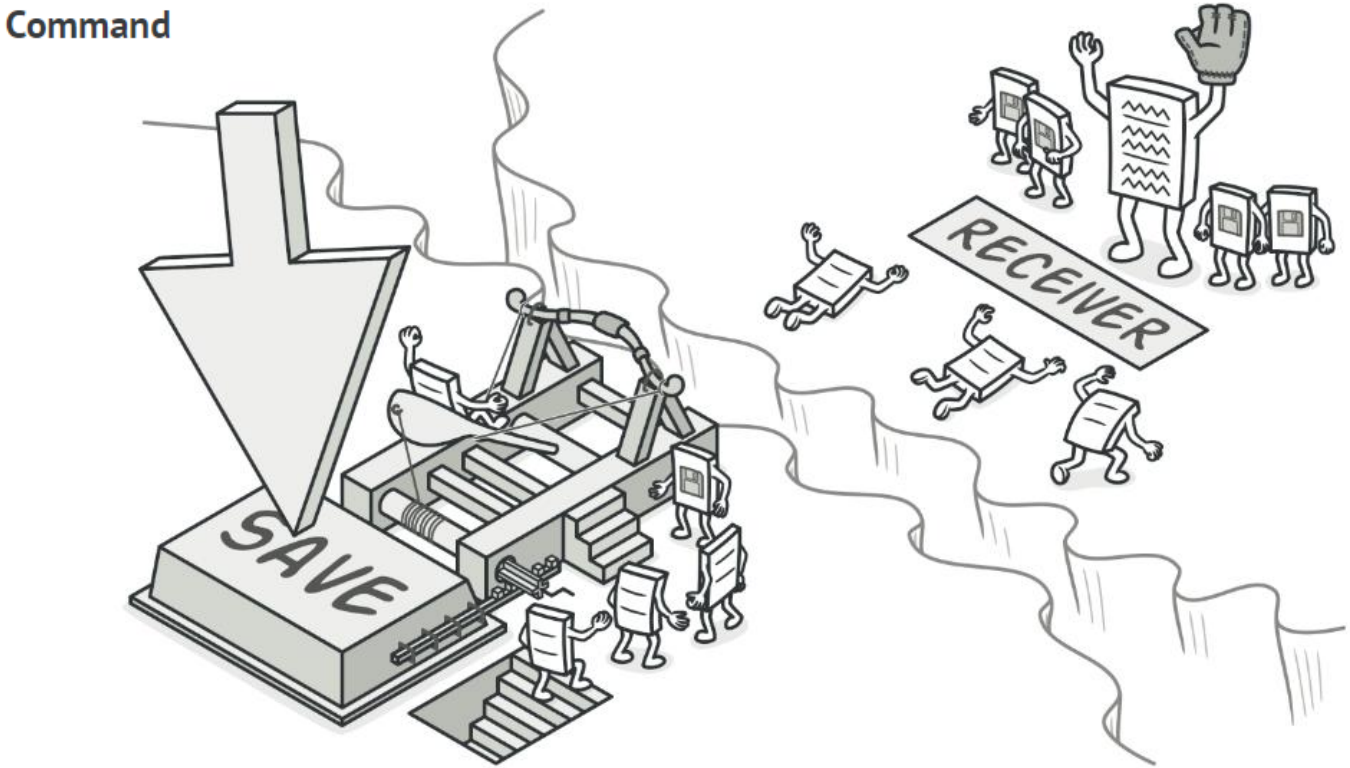
- Las peticiones emitidas por un objeto deben ser atendidas por distintos objetos receptores.
- No se sabe a priori cual es el objeto que me puede resolver el problema.
- Cuando un pedido debe ser manejado por varios objetos.
- El conjunto de objetos que pueden tratar una petición debería ser especificado dinámicamente.



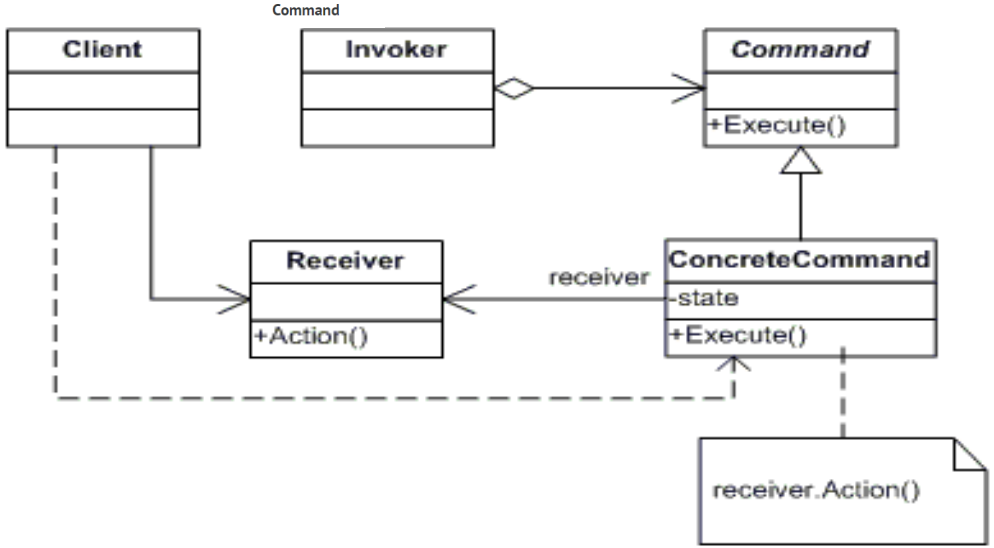
Command



Permite solicitar una operación a un objeto sin conocer realmente el contenido de esta operación, ni el receptor real de la misma.



Command



Command: declara un interface para la ejecución de una operación

ConcreteCommand: define un vínculo entre un objeto Receiver y una acción, implementa Execute() invocando al método correspondiente de Receiver

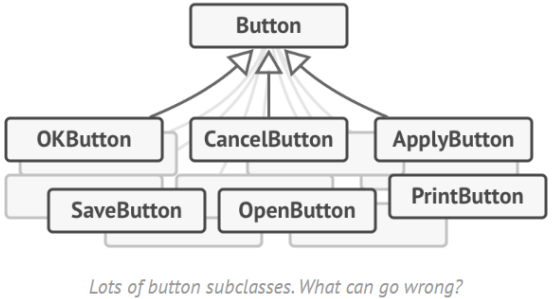
Cliente:Crea un objeto ConcreteCommand y establece su receptor

Invoker:pide a Command que lleve a cabo su petición

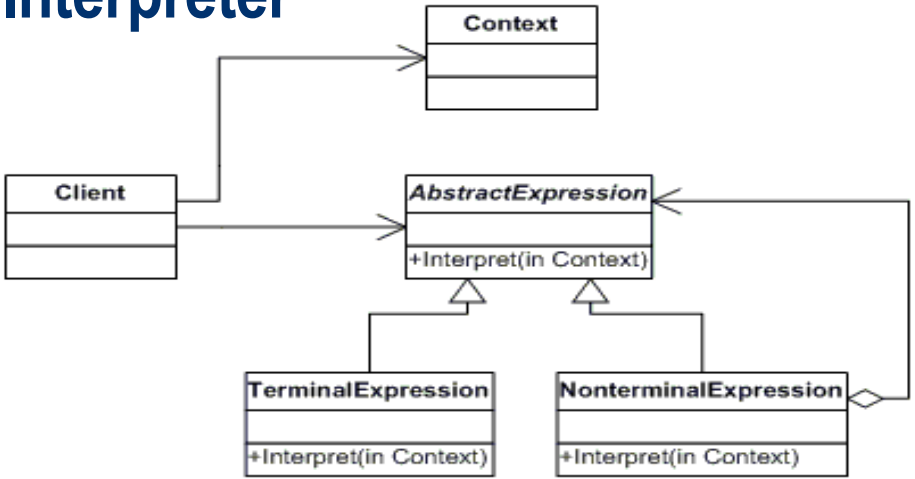
Receiver:sabe cómo realizar las operaciones asociadas con la puesta en marcha de la petición

Se debe usar cuando:

- Se necesiten colas o registros de mensajes.
- Se deba tener la posibilidad de deshacer las operaciones realizadas.
- Se necesite uniformidad al invocar las acciones.
- Se quiera facilitar la parametrización de las acciones a realizar.
- Se quiera independizar el momento de petición del de ejecución.
- El parámetro de una orden puede ser otra orden a ejecutar.
- Se busque desarrollar sistemas utilizando órdenes de alto nivel que se construyen con operaciones sencillas (primitivas).
- Se necesite sencillez al extender el sistema con nuevas acciones.



Interpreter



Dado un lenguaje, define una representación para su gramática junto con un intérprete del lenguaje.

AbstractExpression: declara un interfaz para la ejecución de una operación

TerminalExpression:Implementa una operación de interpretación asociada con símbolos terminales asociados con la gramática

NonterminalExpression:implementa una operación de interpretación asociada con los símbolos no terminales asociados con la gramática

Context:contiene información global para el interprete

Client:construye un árbol sintáctico abstracto que representa una sentencia particular en el lenguaje que la gramática define

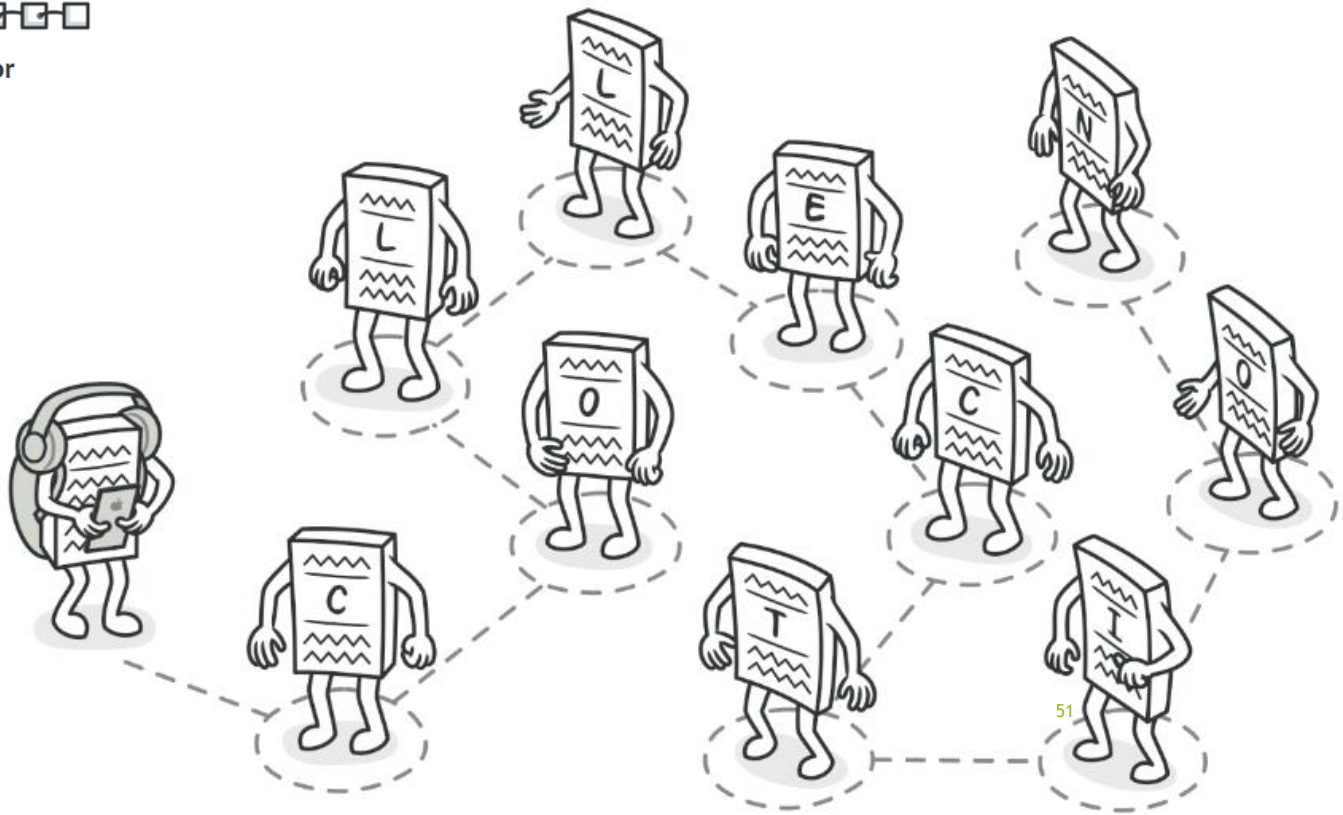
Cuando Utilizarlo.

Este patrón se debe utilizar cuando hay un lenguaje que interpretar y se puede interpretar sus palabras como árboles sintácticos abstractos. Para ello, la gramática debe ser simple.
Difícilmente el desarrollador utilice este patrón en algún momento de su vida, lo que no quita que no sea un patrón utilizado. La situación ideal que se debe considerar para aplicar este patrón es que exista un lenguaje sencillo que pueda interpretarse con palabras. El ejemplo más claro es JAVA: este lenguaje permite escribir en archivos .java entendibles por humanos y luego este archivo es compilado e interpretado para que pueda ejecutar sentencias entendibles por una máquina.

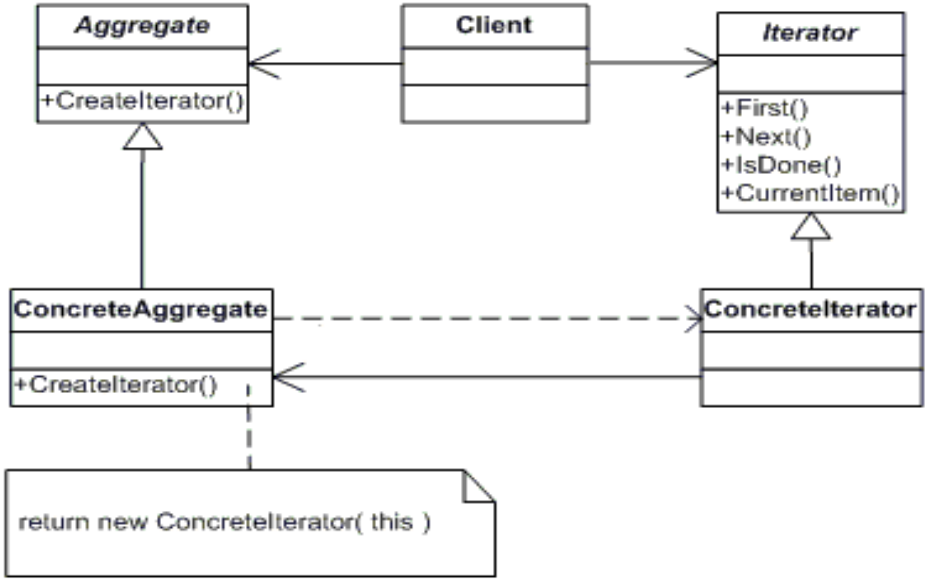
ITERATOR



Este patrón define una interfaz que declara métodos para el acceso secuencial de objetos en una colección. Una clase que acceda una colección únicamente a través de esta interfaz, se mantiene independiente de la clase que implementa la interfaz.



Iterator



Iterator:define una interfaz para atravesar y acceder a los elementos

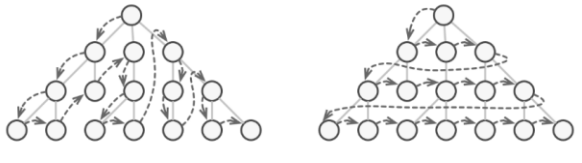
ConcreteIterator:implementa la interfaz del iterator, mantiene un puntero al conjunto de elementos

Aggregate:define un interfaz para crear un objeto iterator

ConcreteAggregator: Implementa la interfaz de creación del Iterator devolviendo un objeto de ConcreteIterator apropiado.

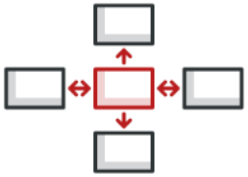
Se utiliza cuando:

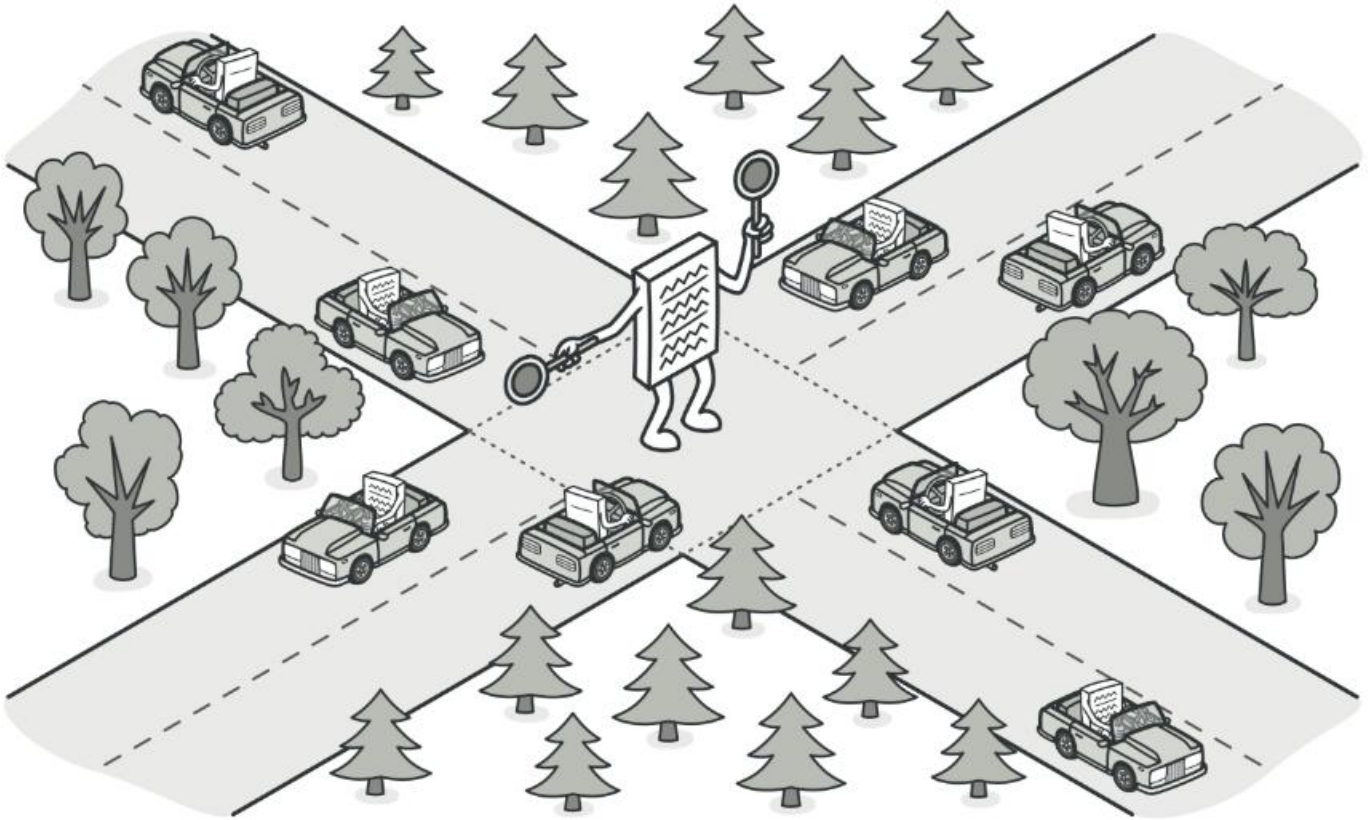
- Una clase necesita acceder al contenido de una colección sin llegar a ser dependiente de la clase que es utilizada para implementar la colección, es decir sin tener que exponer su representación interna.
- Una clase necesita un modo uniforme de acceder al contenido de varias colecciones.
- Cuando se necesita soportar múltiples recorridos de una colección.



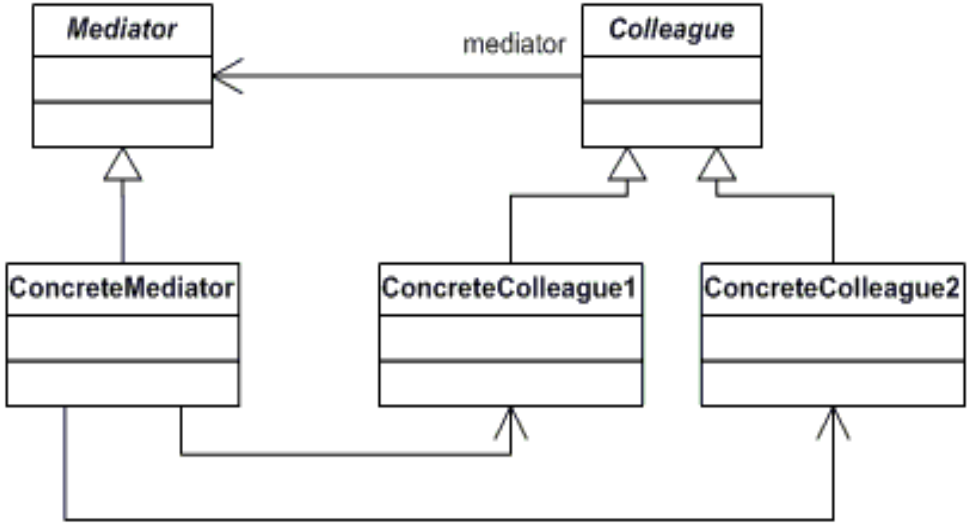
Mediator

Define un objeto que encapsula cómo un conjunto de objetos interactúan.





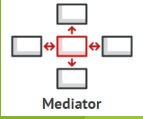
Mediator



Mediator:define una interfaz para la comunicación entre objetos Colleagues

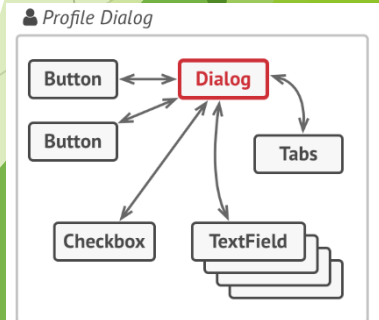
ConcreteMediator:Implement a un comportamiento cooperativo coordinando objetos Colleagues

Colleague classes:cada objeto Colleague conoce su objeto Mediator, cada colleague comunica con su mediador cuando tiene que comunicar con otro colleague

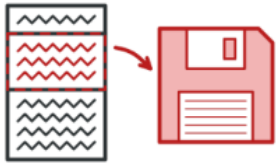


Usar el patrón Mediator cuando:

- Un conjunto grande de objetos se comunica de una forma bien definida, pero compleja.
- Reutilizar un objeto se hace difícil por que se relaciona con muchos objetos.
- Las clases son difíciles de reutilizar porque su función básica esta entrelazada con relaciones de dependencia.



Memento



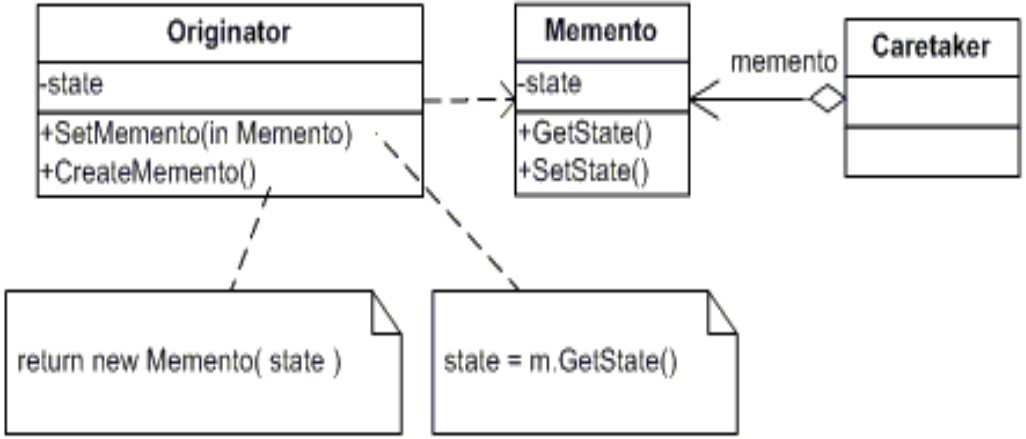
Memento

Se usa cuando:

- Se necesite restaurar el sistema desde estados pasados.
- Se quiera facilitar el hacer y deshacer de determinadas operaciones, para lo que habrá que guardar los estados anteriores de los objetos sobre los que se opere (o bien recordar los cambios de forma incremental).



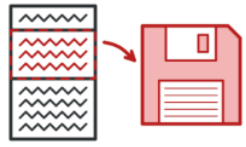
Memento

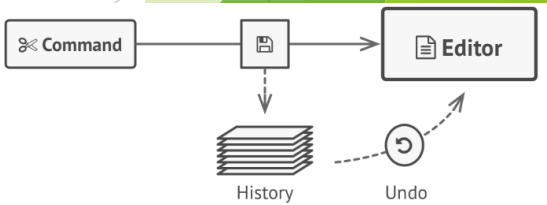


Memento:Almacena el estado interno del objeto Originator, protege del acceso por parte de objetos distintos al Originator

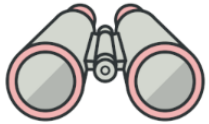
Originator:crea un objeto memento, que contiene una foto fija de su estado interno

Caretaker:es el responsable de mantener la seguridad del objeto memento, no opera o examina el contenido de memento



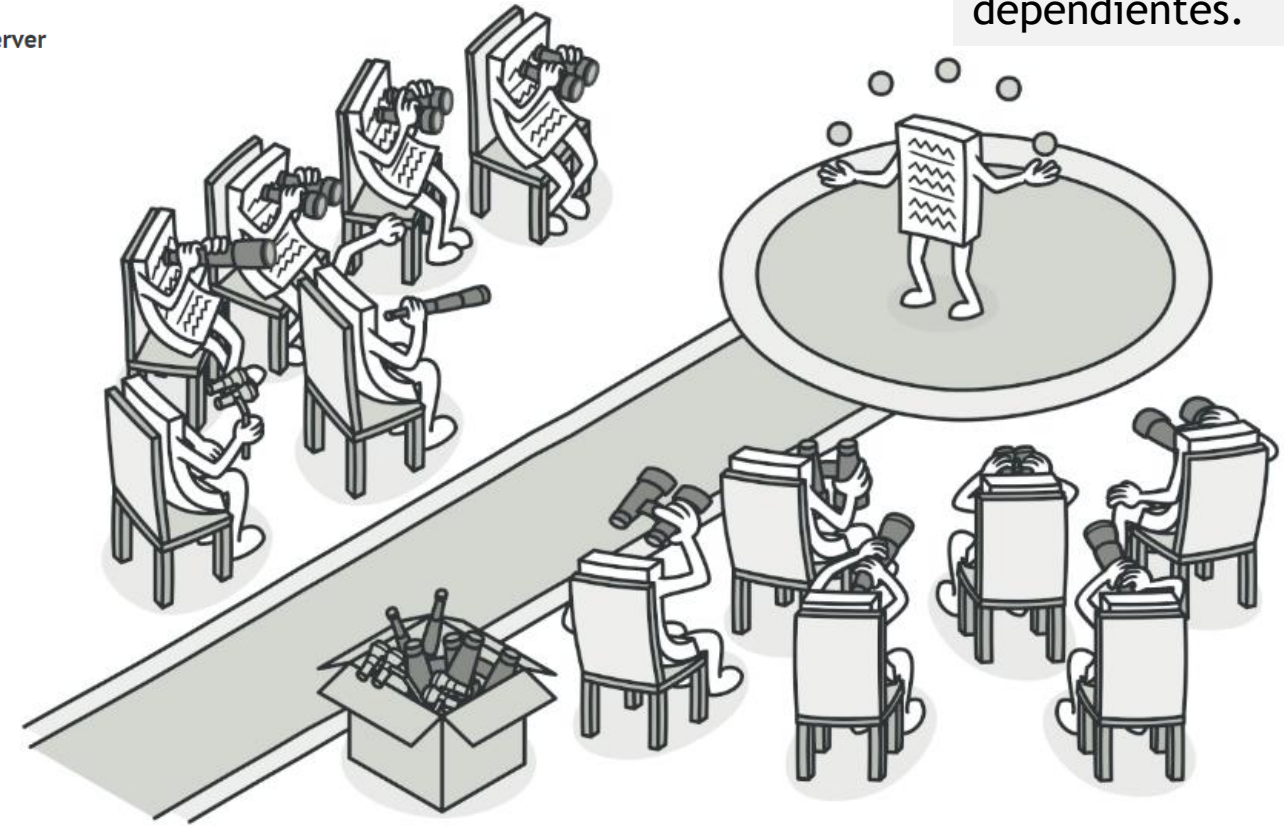


Observer

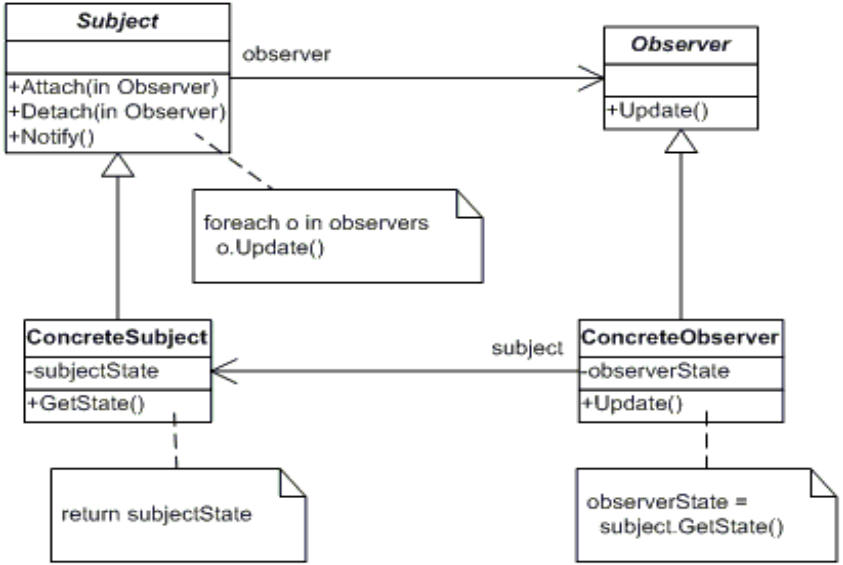


Observer

Define una dependencia del tipo uno a muchos entre objetos, de manera que cuando uno de los objetos cambia su estado, notifica este cambio a todos los dependientes.



Observer





Subject:conoce sus observadores y proporciona una interfaz para gestionarlos

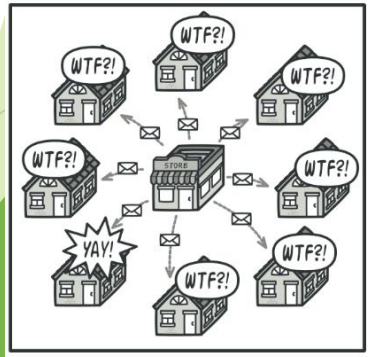
ConcreteSubject: objeto que cambia de estado. Almacena el estado de interés y envía una notificación a sus observadores cuando su estado cambia

Observer: define una interfaz para los objetos a los que debe notificarse el cambio de estado en ConcreteSubject

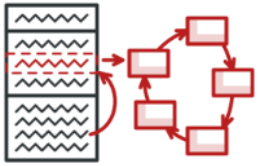
ConcretObserver: mantiene una referencia a un objeto ConcreteSubject, mantiene información consistente relacionada con el estado implementando el método Update()

Debe ser utilizado cuando:

- Un objeto necesita notificar a otros objetos cuando cambia su estado. La idea es encapsular estos aspectos en objetos diferentes permite variarlos y reutilizarlos independientemente.
- Cuando existe una relación de dependencia de uno a muchos que puede requerir que un objeto notifique a múltiples objetos que dependen de él cuando cambia su estado.

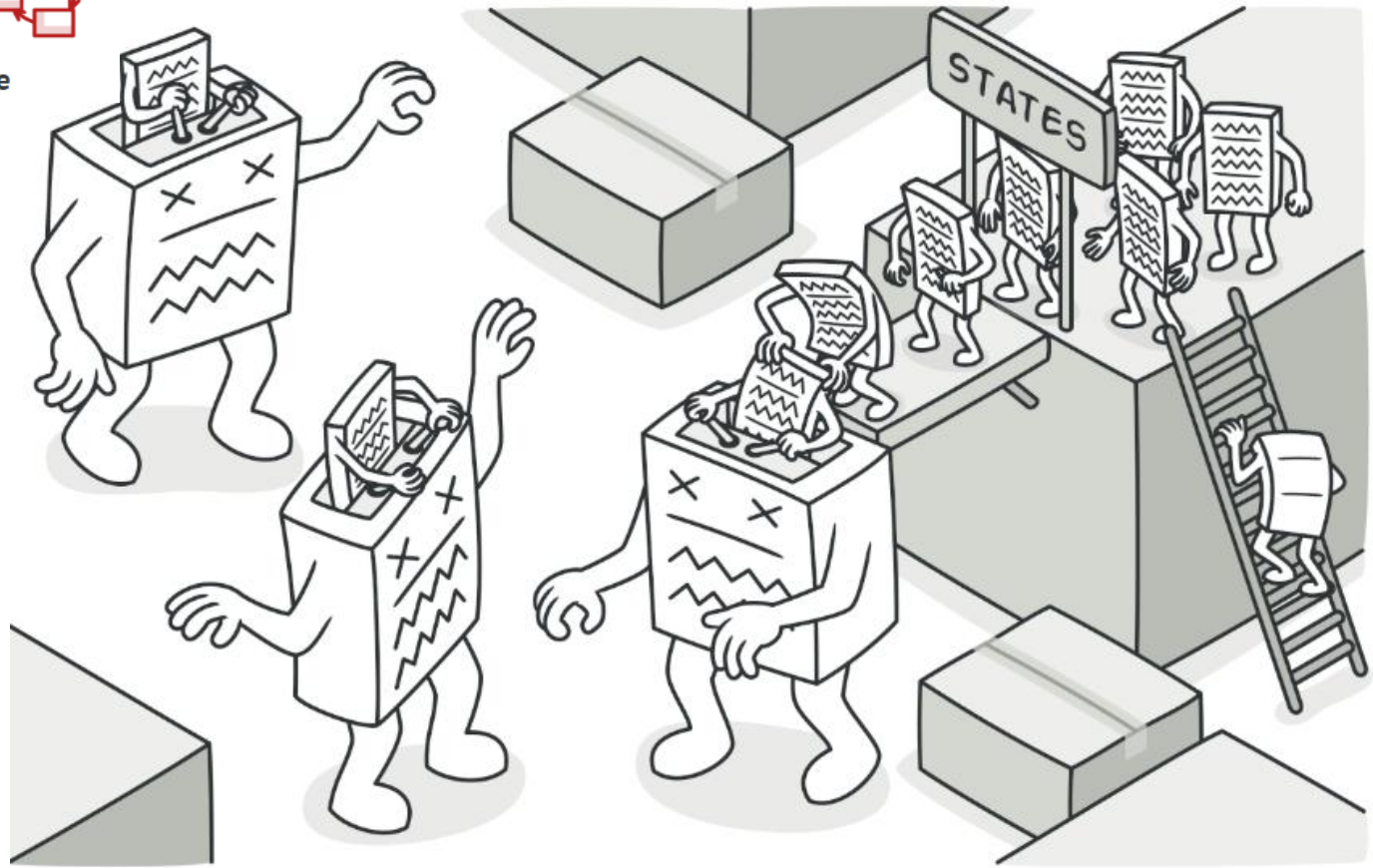


State

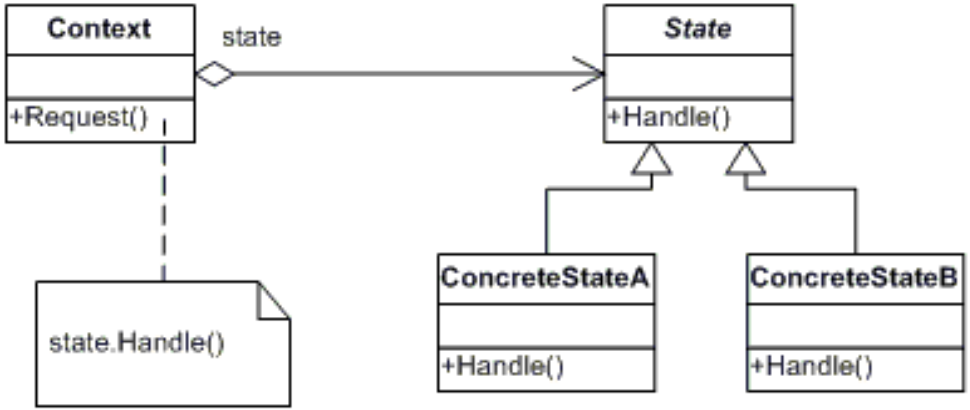


State

Se utiliza cuando el comportamiento de un objeto cambia dependiendo del estado del mismo.



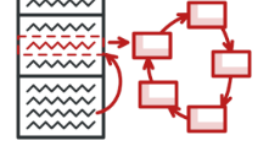
State



Context:define la interfaz de interés a clientes

State:define una interfaz para encapsular el comportamiento asociado con un estado particular del contexto

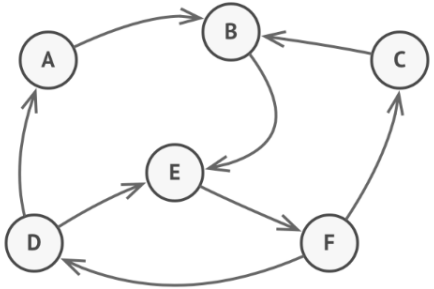
ConcreteState:cada subclase implementa un comportamiento asociado con un estado del Context



Este patrón debe ser utilizado cuando:

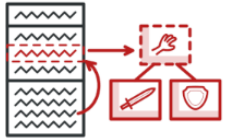
- El comportamiento de un objeto depende de un estado, y debe cambiar en tiempo de ejecución según el comportamiento del estado.
- Cuando las operaciones tienen largas sentencias con múltiples ramas que depende del estado del objeto.

Este patrón se utiliza cuando un determinado objeto tiene diferentes estados y también distintas responsabilidades según el estado en que se encuentre en determinado instante. También puede utilizarse para simplificar casos en los que se tiene un complicado y extenso código de decisión que depende del estado del objeto.



Finite-State Machine.

Strategy

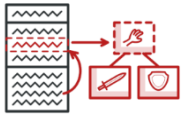


Strategy

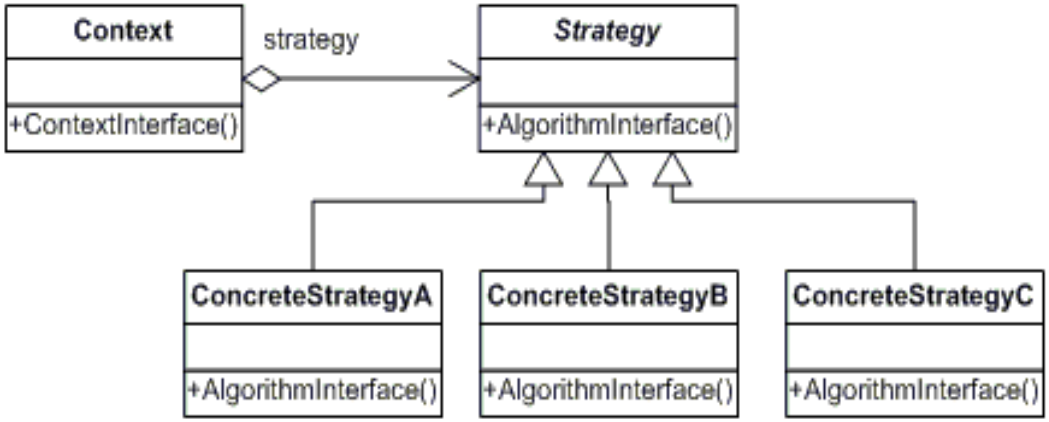
Permite mantener un conjunto de algoritmos de entre los cuales el objeto cliente puede elegir aquel que le conviene e intercambiarlo dinámicamente según sus necesidades.



Strategy



Strategy



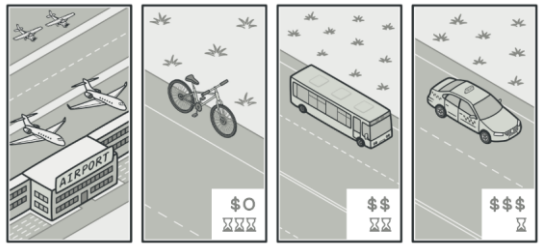
Strategy:declara un interfaz común para dar soporte a todos los algoritmos. Context utiliza esta interfaz para llamar al algoritmo definido por un ConcreteStrategy

ConcreteStrategy: Implementa el algoritmo usando la interfaz Strategy

Context:se configura con un objeto ConcreteStrategy, sobre el que mantiene una referencia, puede definir una interfaz que permita a Strategy acceder a sus datos.

Este patrón debe utilizarse cuando:

- Un programa tiene que proporcionar múltiples variantes de un algoritmo o comportamiento.
- Es posible encapsular las variantes de comportamiento en clases separadas que proporcionan un modo consistente de acceder a los comportamientos.
- Permite cambiar o agregar algoritmos, independientemente de la clase que lo utiliza.



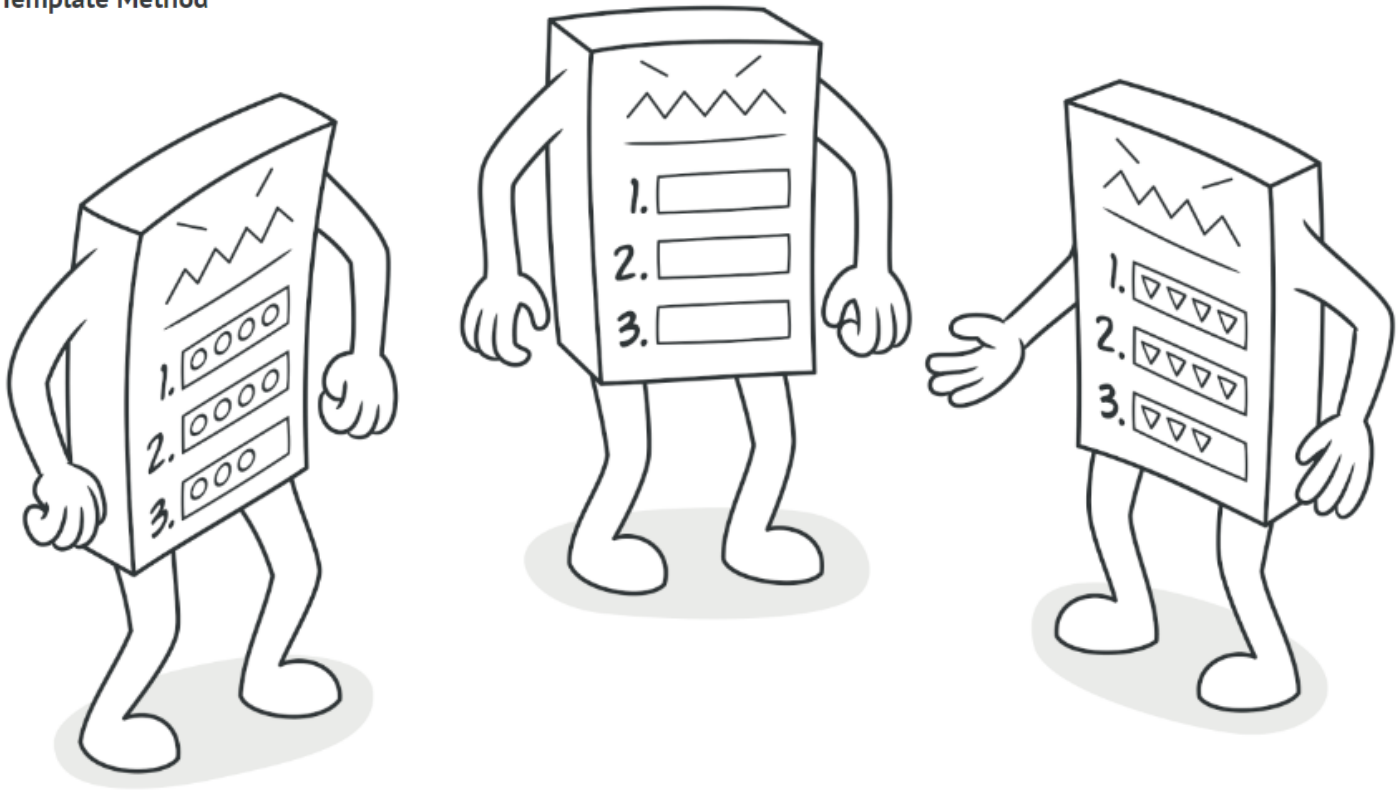
Various strategies for getting to the airport.

Template Method

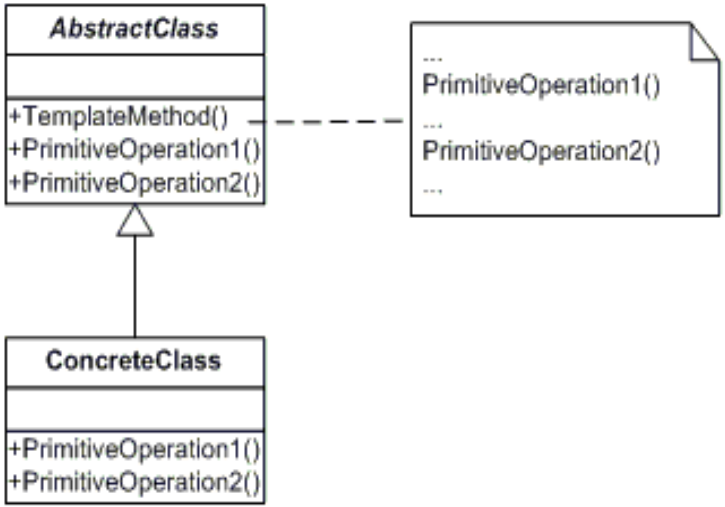


Template Method

Permite redefinir ciertos pasos seguros de un algoritmo sin cambiar la estructura del algoritmo.

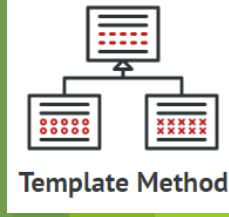


Template Method



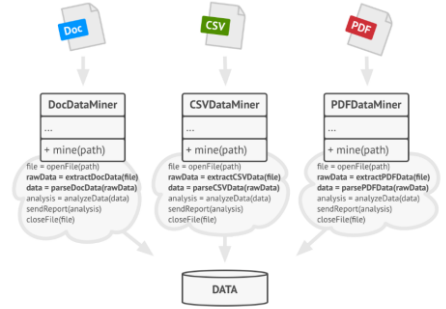
AbstractClass:define operaciones primitivas abstractas cuya concreción se delega a las subclases, estas primitivas se utilizan en el cuerpo de un algoritmo esqueleto

ConcreteClass: implementa las operaciones primitivas abstractas anteriores



Se debe utilizar este patrón cuando.

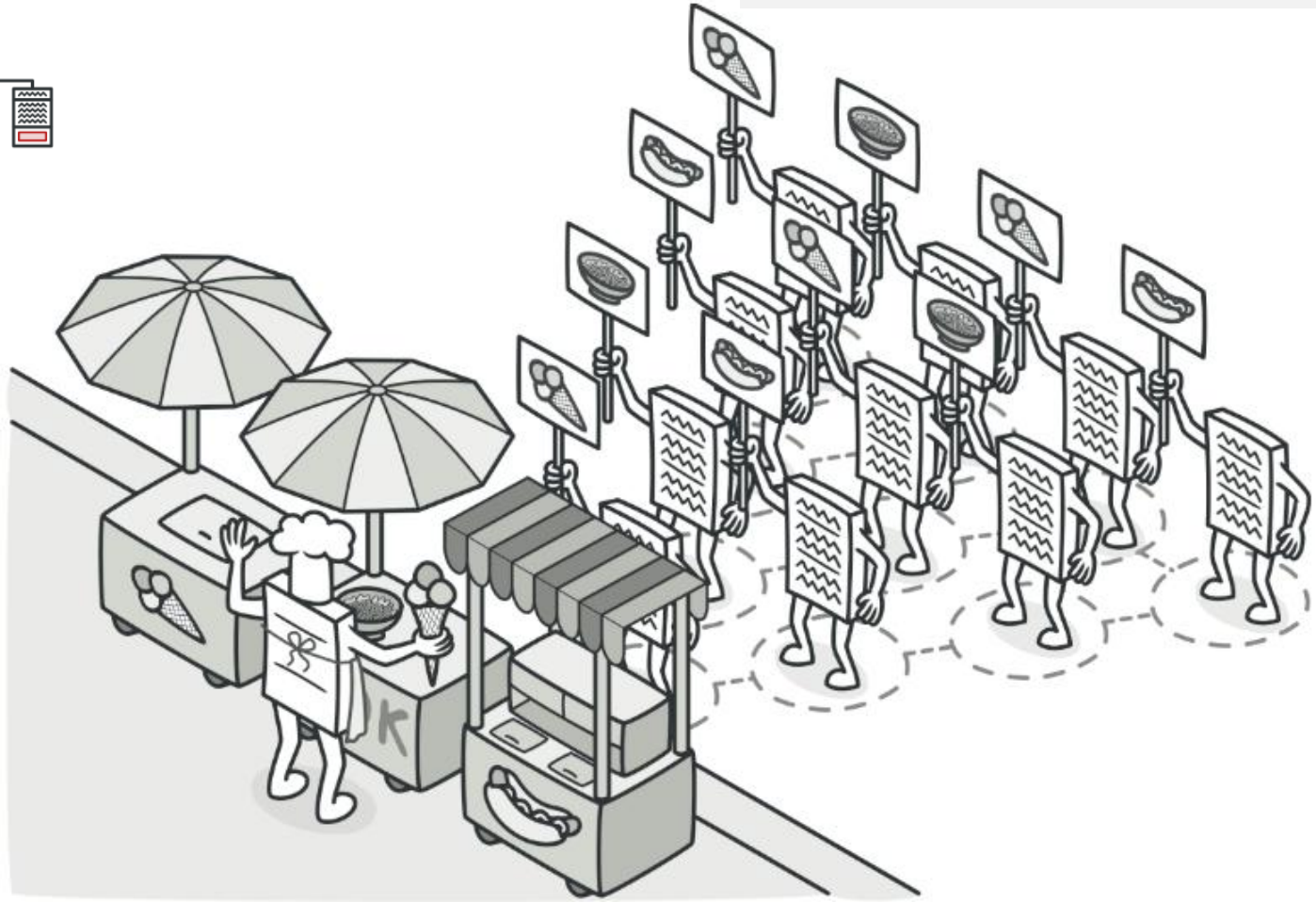
- Se quiera factorizar el comportamiento común de varias subclases.
- Se necesite implementar las partes fijas de un algoritmo una sola vez y dejar que las subclases implementen las partes variables.
- Se busque controlar las ampliaciones de las subclases, convirtiendo en métodos plantillas aquéllos métodos que pueden ser redefinidos.



Visitor

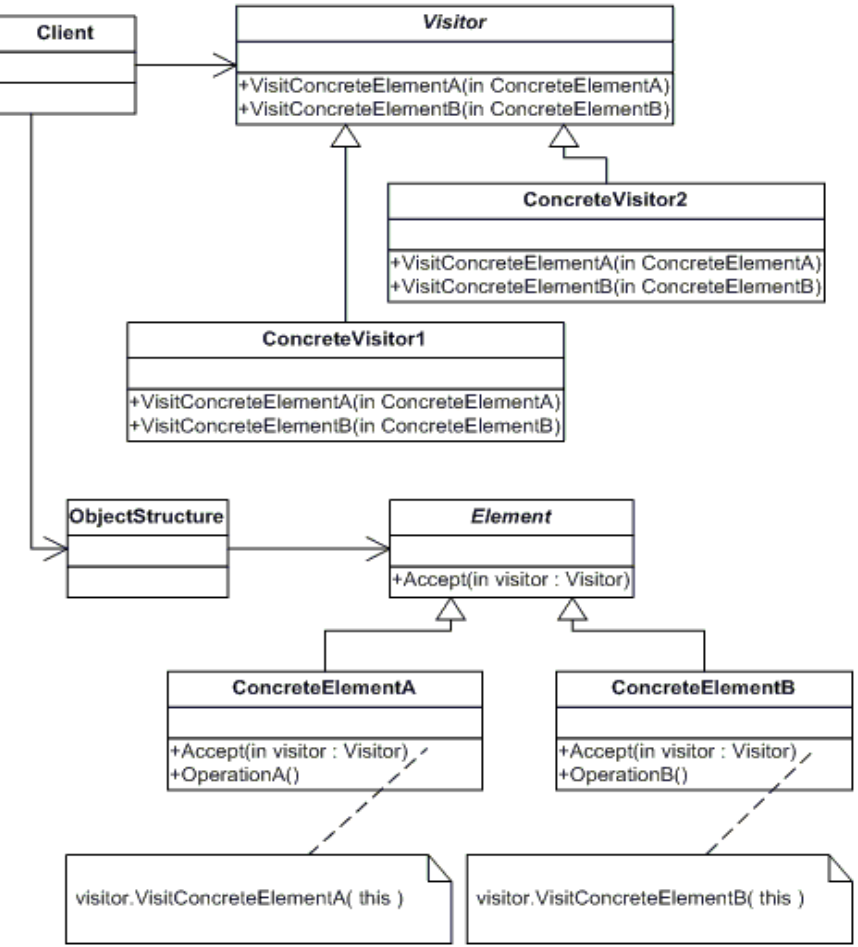
Es una forma de separar el algoritmo de la estructura de un objeto.





Visitor





Este patrón debe utilizarse cuando.

- Una estructura de objetos contiene muchas clases de objetos con distintas interfaces y se desea llevar a cabo operaciones sobre estos objetos que son distintas en cada clase concreta.
- Se quieren llevar a cabo muchas operaciones dispares sobre objetos de una estructura de objetos sin tener que incluir dichas operaciones en las clases.

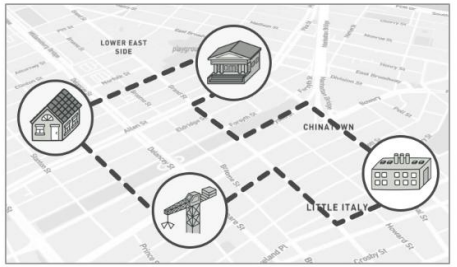
Visitor:Establece las operaciones a realizar

ConcreteVisitor: implementa dichas operaciones

Element: define un método Accept() que toma un Visitor como argumento

ConcreteElement: implementa el método Accept()

ObjectStructure:

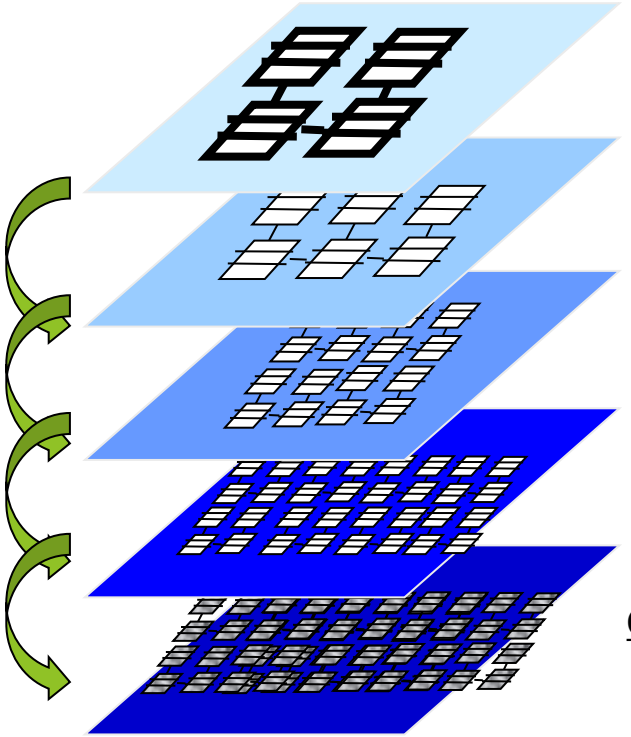


Exporting the graph into XML.

Patrones en el diseño

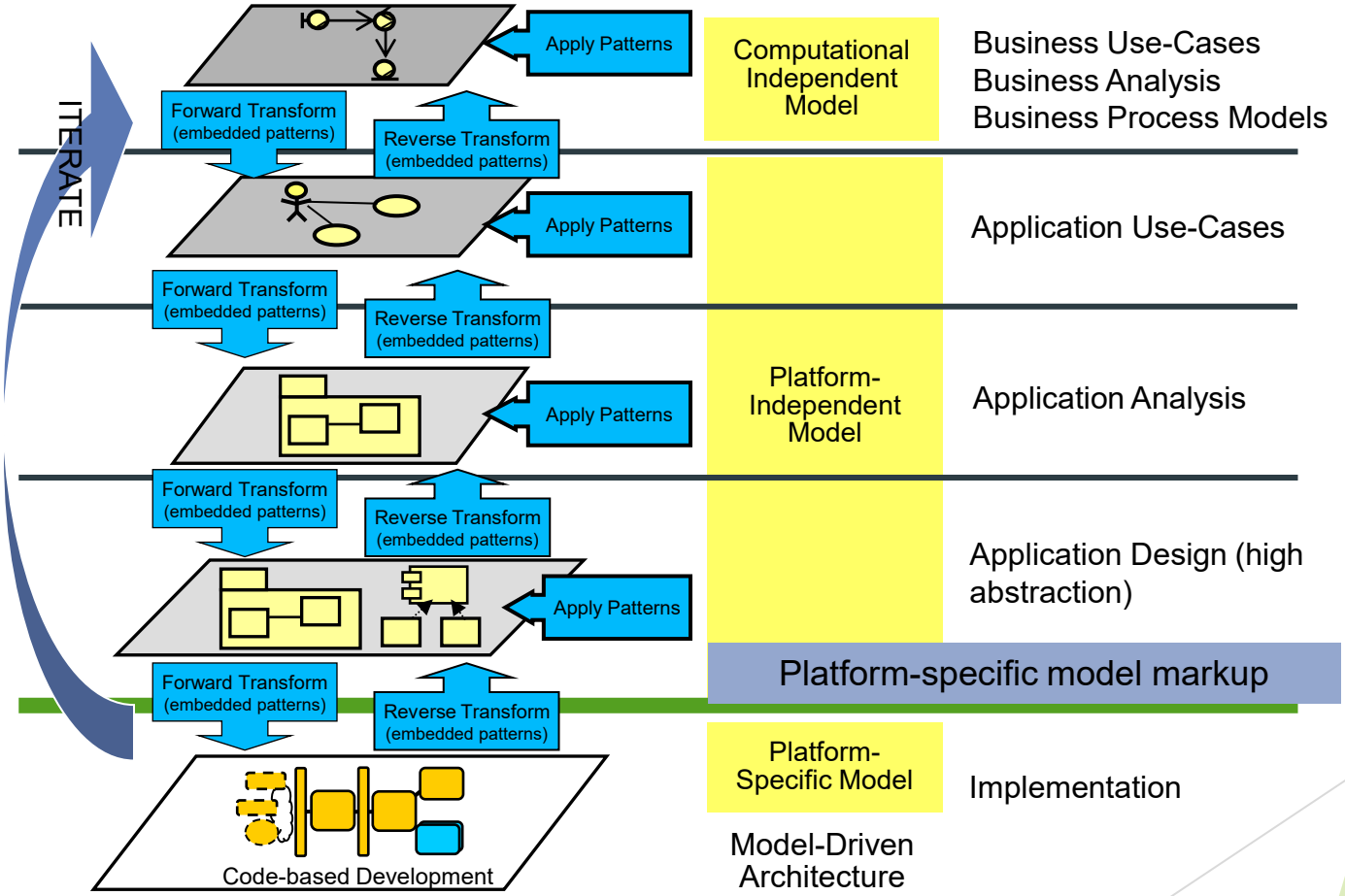
Design decisions made using models.

Patterns automate the transitions from models to working solution.



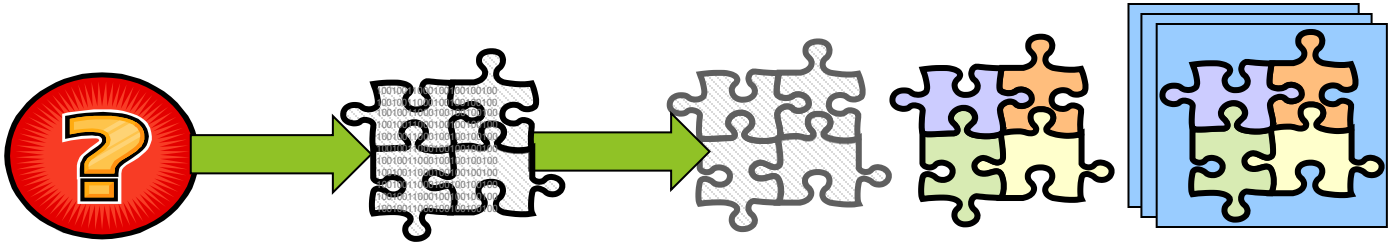
High-level model

Code artifacts


ITERATE
Apply Patterns
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Apply Patterns
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Apply Patterns
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Apply Patterns
Forward Transform (embedded patterns)
Reverse Transform (embedded patterns)
Code-based Development
Computational Independent Model
Business Use-Cases
Business Analysis
Business Process Models
Application Use-Cases
Platform-Independent Model
Application Analysis
Application Design (high abstraction)
Platform-specific model markup
Platform-Specific Model
Implementation
Model-Driven Architecture

No es obligatorio utilizar los patrones en la solución que se esta modelando, solo es aconsejable aplicar el patrón en el caso de tener el mismo problema que ya tenga una solución probada“

Abusar o forzar el uso de los patrones puede ser un error".


Problema
Ejemplo
(Solución)
Especificación
Patron
Implementación
Patron
Patrón

Fuentes

- Design Patterns
- Erich Gamma, Richard Helm, Ralph Johnson y John Vlisides



